S.no	**Topic**	Subtopic (Project work included)
5	**Node JS**	**1)Introduction to Node.js** **2)Setting up Node.js Environment** **3)Basic Modules in Node.js** **4)Package Management with npm** **5)Express.js Framework** **6)Setting up an Express Server** **7)Routing in Express** **8)Middleware in Express** **9)Handling Requests and Responses** **10)Database Integration** **11)Introduction to MongoDB** **12)Setting up MongoDB with Mongoose** **13)CRUD Operations with MongoDB** **14)Authentication and Authorization** **15)Understanding JWT (JSON Web Tokens)** **16)Using Passport.js for Authentication** **17)API Development** **18)RESTful APIs** **19)Versioning and Documentation of APIs** **20)Real-time Communication** **21)Introduction to WebSockets** **22)Using socket.io** **23)Building a Real-time Chat Application** **24) Deployment of Backend API's using Render.com**

1. Introduction to Node js

Node.js is an open-source, cross-platform, server-side runtime environment that executes JavaScript code outside of a browser.

NodeJS is a runtime environment for executing JavaScript outside the browser, **built on the V8 JavaScript engine.** It enables server-side development, supports asynchronous, event-driven programming, and efficiently handles scalable network applications.

- **NodeJS is single-threaded, utilizing an event loop** to handle multiple tasks concurrently.
- It **is asynchronous and non-blocking**, meaning operations do not wait for execution to complete.
- The **V8 engine compiles JavaScript to machine code**, making NodeJS fast and efficient.

1.1 Traditional Server vs Node.js Server

Traditional Threaded Model

- Each client connection → a new thread
- Thread pool is limited → requests may be blocked
- High memory and CPU usage

Node.js Event-driven Model

- Single thread handles all requests
- Delegates I/O to OS threads
- Non-blocking event loop handles responses
- Result: More scalability, lower memory usage, and faster response time

1.2 How NodeJS Works?

NodeJS operates on a single thread but efficiently handles multiple concurrent requests using an event loop.

- **Client Sends a Request:** The request can be for data retrieval, file access, or database queries.
- **NodeJS Places the Request in the Event Loop:** If the request is non-blocking (e.g., database fetch), it is sent to a worker thread without blocking execution.
- **Asynchronous Operations Continue in Background:** While waiting for a response, NodeJS processes other tasks.
- **Callback Execution:** Once the operation completes, the callback function executes, and the response is sent back to the client.

1.3 Core Features

Feature	Description
Event-driven & non-blocking	Efficient handling of many requests using a single thread
Asynchronous I/O	Reads/writes from files, DBs, or networks without blocking
Single-threaded	No thread-per-request; avoids context-switching overhead
Built-in modules	HTTP, File System, OS, Path, etc. available out of the box
JavaScript everywhere	Same language on frontend and backend

1.4 Where to Use NodeJS?

NodeJS is best suited for applications that require high performance, scalability, and real-time processing. Below are some common use cases:

- **Web APIs and Backend Services:-** Ideal for building **Restful API** and **GraphQL APIs**. It also Used in backend services for mobile apps and web applications.
- **Real-Time Applications:** Chat applications (e.g., WhatsApp, Slack). Live streaming services (e.g., Netflix, Twitch).

- **Microservices Architecture:** It helps in developing scalable and independent services. It also used in cloud-based applications.
- **IoT (Internet of Things) Applications:** Handles real-time data streaming from IoT devices. It is suitable for smart home automation and sensor-based systems.
- **Serverless Computing:** Works well with **AWS Lambda, Azure Functions, and Google Cloud Functions.** Runs lightweight serverless functions efficiently.
- **Single-Page Applications (SPAs):** It used in React, Angular, and Vue.js applications. It manages API requests efficiently in the backend.
- **Data-Intensive Applications:** It used for big data processing and real-time analytics. It works well with NoSQL databases like MongoDB and Firebase.

1.5 Use Cases of NodeJS

Here are some use cases of NodeJS:

- **Real-Time Applications:** Ideal for chat platforms, gaming, and collaborative tools that require instant data updates.

- **API Development:** Efficiently handles multiple simultaneous requests, making it suitable for building RESTful APIs.
- **Single-Page Applications (SPAs):** Supports dynamic content loading without full page reloads, enhancing user experience.

1.6 Node.js Versioning

Node.js uses **Semantic Versioning (SemVer)** format:

MAJOR.MINOR.PATCH

Example: **v20.11.0**

Segment	Description
MAJOR	Breaking changes
MINOR	Backward-compatible features
PATCH	Bug fixes only

1.7 LTS vs Current Versions

LTS version stands for the **Long term Support** version, where the release of the software is maintained for an extended period. The LTS version is commonly recommended for most users due to its stability and extended support.

The versions are divided into three main types:

- **Current:** The latest version with new features and improvements. It is updated frequently but is not recommended for production use.
- **Active LTS:** A stable version that receives critical bug fixes, security updates, and non-breaking improvements. This is recommended for production environments.
- **Maintenance LTS:** A version that is in maintenance mode, receiving only critical bug fixes and security updates. It eventually reaches its end-of-life (EOL) status.

Version Type	Purpose	Stability	Recommended for
LTS	Long-Term Support (18 months active + 12 months maintenance)	Very stable	Production apps
Current	Latest features and updates	Less stable	Testing,learning, exploration

You can check installed versions using:

- node -v **# check Node version**
- npm -v **# check npm version**
- nvm ls **# if using Node Version Manager (nvm)**

2.Setting up the node js environment

To start building Node.js applications, you need a development environment where you can:

- Run JavaScript code outside the browser
- Install packages via npm
- Use tools like Express, Mongoose, etc.
- Create, test, and deploy Node-based projects

Versions	Updates	Release Date
22.2.0	More Clear and Concise Documentation Multiple bug fixes to enhance stability and reliability.	**August 2024**
21.7.3	Patched multiple vulnerabilities to strengthen security. Optimizations in runtime execution and garbage collection processes. Enhanced diagnostics and debugging tools.	**July 2024**
20.13.1	Enhanced WebAssembly support. Improvements in the experimental fetch API. New features in the diagnostics report.	**April 2024**

Versions	Updates	Release Date
18.20.2	Updated npm, libuv, and llhttp for improved security and stability. Addressed multiple vulnerabilities to enhance security. Numerous fixes to improve stability and reliability.	**February 2024**
16.20.0	Various dependencies have been updated, enhancing security and stability. Improved support for Apple Silicon (ARM64) builds. Optimizations in the core runtime and garbage collection for better performance and efficiency.	**May 2020**
14.0.0	CLI, report: move --report-on-fatal error to stable deps: upgrade to libuv 1.37.0 fs: add fs/promises alias module	**April 2020**
12.0.0	implement os.type() using uv_os_uname() add a welcome message in repl use ES6 class inheritance style	**April 2019**
10.0.0	Async hooks API has been removed. Console.table() has been added. The file system has been improved Processing of HTTP Status codes 100 , 102-199 has been improved.	**April 2018**

Versions	Updates	Release Date
	Multi-byte characters in URL paths are now forbidden.	
8.0.0	Async hooks landed in the core. npm client has been updated to 5.0.0 Native promise instances are now Domain-aware. Experimental support for the new N-API API has been added REPL magic mode has been deprecated	**May 2017**
6.0.0	A New Buffer Constructor has been added Improved Error handling. Added API to query plain DNS PTR records. 'client error' can now be used to return custom errors from an HTTP server	**April 2016**
4.0.0	**npm:** Upgrade to version 2.14.2 from 2.13.3, includes a security update. Improved Timer Performance. util function has been deprecated in this version. node-gyp updated.	**September 2015**
0.12.0	Sockets Limit increased to infinity Now cluster has two modes of operations	**February 2015**

Versions	Updates	Release Date
	Introduced new **TLSWrap** Mechanism More Accurate mechanism to allocate buffer memory Added API's for load custom engines.	
0.10.0	**npm:** Upgrade to 1.2.14 **core:** Append filename properly in dlopen on windows (isaacs) **zlib:** Manage flush flags appropriately (isaacs) **domains:** Handle errors thrown in nested error handlers (isaacs) **buffer:** Strip high bits when converting to ascii (Ben Noordhuis) **win/msi:** Enable modify and repair (Bert Belder) **win/msi:** Add feature selection for various Node parts (Bert Belder) **win/msi:** use consistent registry key paths (Bert Belder) **child_process:** support sending dgram socket (Andreas Madsen) **fs:** Raise EISDIR on Windows when calling fs.read/write on a dir (isaacs) **unix:** fix strict aliasing warnings, macro-ify functions (Ben Noordhuis) **unix:** honor UV_THREADPOOL_SIZE environment var (Ben Noordhuis)	**March 2013**

Versions	Updates	Release Date
	win/tty: fix typo in color attributes enumeration (Bert Belder) **win/tty:** don't touch insert mode or quick edit mode (Bert Belder)	
0.8.0	Node got a lot faster. Node got more stable. You can do stuff with file descriptors again. The cluster module is much more awesome. The domain module was added. The repl is better. The build system changed from waf to gyp. Some other stuff changed, too. Scroll to the bottom for the links to install i	**June 2012**
0.6.0	Native Windows support using I/O Completion Ports for sockets. Integrated load balancing over multiple processes. Better support for IPC between Node instances Improved command line debugger Built-in binding to zlib for compression	**November 2011**

Versions	Updates	Release Date
0.2.0 - 0.4.0	Stability and bug-fixed	**August 2010 - April 2011**
0.1.0	Runtime Environment on V8 Engine	**May 2010**

2.1 Installation

Step 1: Download & Install Node.js

- Download from Official Site:- https://nodejs.org

You'll see two versions:

- LTS (Recommended for most users) – Stable and production-ready
- Current – Latest features, not fully tested

Installation includes:

- Node.js runtime
- npm (Node Package Manager)

Verify Installation

```
node -v   # shows Node version
npm -v    # shows npm version
```

Step 2: Install Code Editor (Recommended https://code.visualstudio.com/

Useful Extensions:

- ESLint
- Prettier
- REST Client
- Node.js Extension Pack
- GitLens

Step 3: Set Up a Project Folder

- Open VS Code
- Create a folder (e.g., nodejs-basics)
- Open terminal in that folder

Run:

```
npm init -y
```

This creates a package.json file which stores: Project metadata, Scripts, Dependencies

Step 4: Write Your First Node.js Script :-

index.js:- console.log("Hello from Node.js!");

Run it using terminal:- **node index.js**

Output: **"**Hello from Node.js!"

Optional: Install Node Version Manager (nvm)
Useful if you want to switch between multiple versions of Node.js.

Install nvm (Linux/macOS):

curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.39.7/install.sh | bash

Then:

- nvm install 18 # installs Node 18
- nvm use 18 # switches to Node 18
- nvm list # shows installed versions

For Windows: Use nvm-windows

2.2. Node JS NPM

NPM (Node Package Manager) is a package manager for NodeJS modules. It helps developers manage project dependencies, scripts, and third-party libraries. By installing NodeJS on your system, NPM is automatically installed, and ready to use.

- It is primarily used to manage packages or modules—these are pre-built pieces of code that extend the functionality of your NodeJS application.
- The NPM registry hosts millions of free packages that you can download and use in your project.
- NPM is installed automatically when you install NodeJS, so you don't need to set it up manually.

2.3. Package in Node JS

A package in NodeJS is a reusable module of code that adds functionality to your application. It can be anything from a small utility function to a full-featured library.

- Packages can be installed from the NPM registry.
- They are stored in the node_modules folder in your project.
- You can easily install, update, or remove packages with NPM commands.

2.4. Managing Project Dependencies

1. Installing All Dependencies:- In a NodeJS project, dependencies are stored in a package.json file. To install all dependencies listed in the file, run:

```
npm install
```

This will download all required packages and place them in the node_modules folder.

2. Installing a Specific Package:- To install a specific package, use:

```
npm install <package-name>
```

You can also install a package as a development dependency using:

```
npm install <package-name> --save-dev
```

Development dependencies are packages needed only during development, such as testing libraries. To install a package and simultaneously save it in **package.json** file (in case using NodeJS), **add --save flag**. The **--save flag** is **default** in npm install command.

Example:

```
npm install express --save
```

Usage of Flags:

- **--save:** flag one can control where the packages are to be installed.
- **--save-prod :** Using this packages will appear in Dependencies which is also by default.
- **--save-dev :** Using this packages will get appear in devDependencies and will only be used in the development mode.

Note: If there is a package.json file with all the packages mentioned as dependencies already, just type npm install in terminal

3. Updating Packages

You can easily update packages in your project using the following command

```
npm update
```

This will update all packages to their latest compatible versions based on the version constraints in the package.json file.

To update a specific package, run

```
npm update <package-name>
```

4. Uninstalling Packages:- To uninstall packages using npm the below syntax:

```
npm uninstall <package-name>
```

For uninstall Global Packages

```
npm uninstall package_name -g
```

2.5. Popular NPM Packages for NodeJS

NPM has a massive library of packages. Here are a few popular packages that can enhance your NodeJS applications

Packages	Description
Express	A fast, minimal web framework for building APIs and web applications.
Mongoose	A MongoDB object modeling tool for NodeJS.
Lodash	A utility library delivering consistency, customization, and performance.
Axios	A promise-based HTTP client for making HTTP requests.
React	A popular front-end library used to build user interfaces.
Dotenv	Loads environment variables from a .env file into process.env.
Nodemon	Automatically restarts the server during development when file changes are detected.

Packages	Description
Jest	A JavaScript testing framework designed to ensure correctness of any NodeJS code.
Socket.io	Enables real-time, bidirectional communication between web clients and servers.

3. Node JS modules & architecture

Node.js is a JavaScript-based platform mainly used to create I/O-intensive web applications such as chat apps, multimedia streaming sites, etc. It is built on Google Chrome's V8 JavaScript engine.

3.1 Web Applications

A web application is software that runs on a server and is rendered by a client browser that accesses all of the application's resources through the Internet.

A typical web application consists of the following components:

- **Client:** A client refers to the user interacting with the server by sending out requests.
- **Server:** The server is responsible for receiving client requests, performing appropriate tasks, and returning results to the clients. It bridges the front end and the stored data, allowing clients to perform operations on the data.
- **Database:** A database is where a web application's data is stored. The data can be created, modified, and deleted depending on the client's request.

- **VPS servers** offer base capabilities and environments to integrate Node.js apps with developer tools and APIs.

3.2 Node.js Server Architecture

To manage several concurrent clients, Node.js employs a **“Single Threaded Event Loop"** design. The JavaScript **event-based model** and the JavaScript **callback mechanism** are employed in the Node.js Processing Model.
It employs two fundamental concepts:

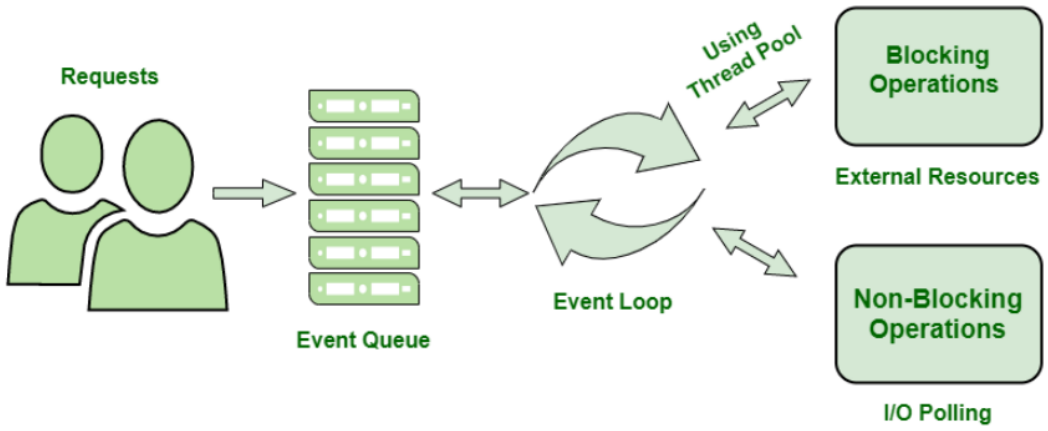
- **Asynchronous model**
- **Non-blocking of I/O operations**

These features enhance the scalability, performance, and throughput of Node.js web applications.

3.3 Components of the Node.js Architecture

- **Requests:** Depending on the actions that a user needs to perform, the requests to the server can be either blocking (complex) or non-blocking (simple).
- **Node.js Server:** The Node.js server accepts user requests, processes them, and returns results to the users.
- **Event Queue:** The main use of Event Queue is to store the incoming client requests and pass them sequentially to the Event Loop.
- **Thread Pool:** The Thread pool in a Node.js server contains the threads that are available for performing operations required to process requests.
- **Event Loop:** Event Loop receives requests from the Event Queue and sends out the responses to the clients.
- **External Resources:** In order to handle blocking client requests, external resources are used. They can be of any type (computation, storage etc).

3.4 Workflow of Node.js Server Architecture



- **Users send requests** (blocking or non-blocking) to the server for performing operations.
- The **requests enter the Event Queue** first at the **server-side**.
- The **Event queue passes** the requests **sequentially to the event loop**. The event loop checks the nature of **the request (blocking or non-blocking).**
- Event Loop processes the non-blocking requests which do not require external resources and returns the responses to the corresponding clients
- For **blocking requests, a single thread** is assigned to the process for completing the task by using external resources.
- After the completion of the operation, the **request is redirected to the Event Loop** which delivers the response back to the client.

3.5 Advantages of Node.js Server Architecture

- The Node.js server can efficiently handle a high number of requests by employing the use of Event Queue and Thread Pool.
- There is no need to establish multiple threads because Event Loop processes all requests one at a time, therefore a single thread is sufficient.
- The entire process of serving requests to a Node.js server consumes less memory and server resources since the requests are handled one at a time.

3.6 Disadvantages of Node.js Server Architecture

Here are the disadvantages of Node.js server architecture in a more concise format:

- **Single-Threaded**: Limited to one thread; can be a bottleneck for CPU-intensive tasks.
- **Callback Hell:** Complex nesting of callbacks can lead to hard-to-maintain code.
- **Performance Bottlenecks:** Not optimal for heavy computational tasks due to non-blocking I/O model.
- **Dependency on Outside Libraries:** Heavy reliance on third-party libraries can impact stability and security.
- **Inconsistent API:** Frequent API changes can lead to backward compatibility issues.
- **Lack of Strong Typing:** JavaScript's lack of strong typing can lead to runtime errors and bugs.

4. Node.js Modules

Modules in Node.js are reusable blocks of code that help organize and structure applications efficiently. Node.js provides a built-in module system that enables developers to use pre-existing functionalities without rewriting code. You can create your own modules or use modules from the **Node.js ecosystem**. This section introduces **different types of modules, how to import and export them**, and how they enhance code modularity and maintainability.

By using modules, developers can keep code organized, reusable, and maintainable. Modules can contain

- **Variables**
- **Functions**
- **Classes**
- **Objects**

4.1 Categories of Modules

Type	Description
Core Modules	Built into Node.js (e.g., `fs`, `path`, `http`)
Local Modules	Your own custom modules
Third-party Modules	Installed via npm (e.g., `express`, `lodash`)

4.2 Common NodeJS Modules

- **HTTP Module:** The http module is used to create web servers. It allows you to handle requests and send responses.
- **FS (File System) Module:** The fs module provides an API to interact with the file system. It can be used to read and write files, check for file existence, etc.
- **Path Module:** The path module helps in handling and transforming file paths. It makes working with file systems easier and more cross-platform.
- **Event Module:** The event module allows objects to emit and listen to events, which helps in writing event-driven applications.

4.2.1 File System (FS) Module

The fs module in Node.js provides an API to interact with the file system in a way that is modeled on standard POSIX functions. You can **read, write, delete, rename**, and manage files & folders programmatically.

There are **two modes** for most functions:

- **Synchronous (blocking)**
- **Asynchronous (non-blocking, preferred)**

4.2.1.1 fs.readFile():- Reading from a file (Asynchronously)

Reads the content of a file asynchronously. Use when performance matters and blocking is not acceptable.

```
fs.readFile('file.txt', 'utf-8', (err, data) => {
  if (err) return console.error(err);
  console.log(data);
});
```

4.2.1.2 fs.readFileSync():- Synchronously

Reads the content of a file synchronously (blocks execution until done). Simple to use, but should be avoided in production for large files or servers.

```
const data = fs.readFileSync('file.txt', 'utf-8');
console.log(data);
```

4.2.1.3 fs.writeFile():- writing into a file (Asynchronously)

Writes data to a file asynchronously. If the file exists, it overwrites the content. Great for saving data like logs, user inputs, or API results.

```
fs.writeFile('output.txt', 'Hello World!', (err) => {
  if (err) return console.error(err);
  console.log('File written successfully!');
});
```

4.2.1.4 fs.writeFileSync():- Synchronously

Blocks the event loop — suitable for CLI scripts, not web servers.

```
fs.writeFileSync('output.txt', 'Sync write example');
```

4.2.1.5 fs.appendFile()

Adds content to the end of a file. Creates the file if it doesn't exist. Ideal for logging, appending histories, or audit trails.

```
fs.appendFile('log.txt', 'New log entry\n', (err) => {
  if (!err) console.log('Data appended!');
});
```

4.2.1.6 fs.appendFileSync()

Synchronous version of appendFile.

```
fs.appendFileSync('log.txt','Appended synchronously\n');
```

4.2.1.7 fs.rename()

Renames or moves a file asynchronously. Can also move files by providing a different path.

```
fs.rename('oldName.txt', 'newName.txt', (err) => {
  if (!err) console.log('File renamed');
});
```

4.2.1.8 fs.renameSync()

Synchronous version of rename.

```
fs.renameSync('old.txt', 'new.txt');
```

4.2.1.9 fs.unlink()

Deletes a file asynchronously. Use with caution — this action is irreversible.

```
fs.unlink('deleteMe.txt', (err) => {
  if (!err) console.log('File deleted!');
});
```

4.2.1.10 fs.unlinkSync()

Synchronous version of unlink.

```
fs.unlinkSync('deleteMe.txt');
```

4.2.1.11. fs.mkdir()

Creates a directory asynchronously. Use recursive: true to create nested folders.

```
fs.mkdir('newFolder', (err) => {
  if (!err) console.log('Folder created!');
});
```

4.2.1.12 fs.mkdirSync()

Synchronous version of mkdir.

```
fs.mkdirSync('newFolder');
```

4.2.1.13. fs.rmdir() (Deprecated, use fs.rm())

Removes a directory **asynchronously**. Must be empty. Use fs.rm(path, { recursive: true }) to remove non-empty folders.

```
fs.rmdir('emptyFolder', (err) => {
  if (!err) console.log('Folder removed!');
});
```

4.2.1.14 fs.readdir()

Reads the contents (files/folders) of a directory. Good for listing files, scanning folders, or building dynamic file explorers.

```
fs.readdir('./', (err, files) => {
  console.log(files); // Returns an array
});
```

4.2.1.15 fs.stat()

Returns metadata/statistics about a file or directory. Use it to check file size, modified date, and type (file/folder).

```
fs.stat('file.txt', (err, stats) => {
  console.log(stats.isFile());
  console.log(stats.size + ' bytes');
});
```

4.2.1.16 fs.existsSync()

Quick check if file/folder exists. This is **deprecated** in async version (fs.exists()), so use fs.access() instead for non-blocking check.

```
if (fs.existsSync('data.txt')) {
  console.log('File exists');
}
```

4.2.1.17. fs.copyFile():- Copies one file to another.

```
fs.copyFile('source.txt', 'copy.txt', (err) => {
  if (!err) console.log('Copied successfully');
});
```

Practical:- fileManager.js

```
const fs = require('fs');

const path = require('path');

// Define paths

const folderPath = path.join(__dirname, 'demo-
folder');

const filePath = path.join(folderPath, 'sample.txt');

const newFilePath = path.join(folderPath,
'renamed.txt');

const copyPath = path.join(folderPath, 'copy.txt');


// Step 1: Create a folder

if (!fs.existsSync(folderPath)) {

  fs.mkdirSync(folderPath);

  console.log('Folder created');

}


// Step 2: Write to a file

fs.writeFileSync(filePath, 'Hello, this is the first
line.\n');

console.log('File created and written');


// Step 3: Append content to the file

fs.appendFileSync(filePath, 'This is an appended
line.\n');

console.log('Content appended');
```

```
// Step 4: Read file (sync)
const syncContent = fs.readFileSync(filePath, 'utf-8');
console.log('Read (sync):\n', syncContent);

// Step 5: Read file (async)
fs.readFile(filePath, 'utf-8', (err, data) => {
  if (!err) {
    console.log('Read (async):\n', data);
  }
});

// Step 6: Rename the file
fs.renameSync(filePath, newFilePath);
console.log('File renamed');

// Step 7: Get file stats
const stats = fs.statSync(newFilePath);
console.log('File Stats:');
console.log(`Size: ${stats.size} bytes`);
console.log(`Created: ${stats.birthtime}`);
console.log(`Is File?`, stats.isFile());
// Step 8: Copy file
fs.copyFileSync(newFilePath, copyPath);
console.log('File copied');

// Step 9: Read files in directory
```

```
const files = fs.readdirSync(folderPath);
console.log('Files in folder:', files);


// Step 10: Delete files
fs.unlinkSync(copyPath);
fs.unlinkSync(newFilePath);
console.log('Original and copied files deleted');


// Step 11: Delete folder
fs.rmdirSync(folderPath);
console.log('Folder deleted');
```

Summary Table

Function	Purpose
readFile()	Read file asynchronously
writeFile()	Write file (overwrite)
appendFile()	Append to file
unlink()	Delete file
rename()	Rename or move file
mkdir()	Create folder
readdir()	Read directory contents
stat()	File/directory info
existsSync()	Check existence (sync only)
copyFile()	Copy file

4.2.2 http Module

The http module allows Node.js to create web servers and handle HTTP requests and responses without using any external library. It uses event-driven architecture. It provides an interface to listen for request events and respond accordingly. It's the backbone for any web application or API built without Express.

Common Use Cases

- Building custom HTTP servers & Learning how web protocols work
- Creating low-level APIs without frameworks

Practical Example: Basic Server

```
const http = require('http');

const server = http.createServer((req, res) => {
  if (req.url === '/') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Home Page');
  } else if (req.url === '/contact') {
    res.writeHead(200, { 'Content-Type': 'text/plain' });
    res.end('Contact Page');
  } else {
    res.writeHead(404, { 'Content-Type': 'text/plain' });
    res.end('404 Not Found');
  }
});

server.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

4.2.3 path Module

The path module provides utilities for working with file and directory paths.

It's especially useful for making your code cross-platform, since path separators differ between Windows (\) and UNIX (/).

Common Use Cases

- Creating dynamic file paths
- Resolving absolute paths
- Extracting parts of a file path

Practical Example

```
const path = require('path');
console.log("Directory Name:", path.dirname(__filename));
console.log("File Name:", path.basename(__filename));
console.log("Extension:", path.extname(__filename));
console.log("Joined Path:", path.join(__dirname, 'data', 'info.txt'));
console.log("Absolute Path:", path.resolve('config', 'app.json'));
```

4.2.4. OS Module

The OS module provides information about the operating system, useful for system-level operations, diagnostics, or adapting behavior based on environment.

It interacts with system resources such as memory, CPU, architecture, and network interfaces.

Common Use Cases

- Logging server metrics
- Environment-based configuration
- Memory monitoring or optimization

Practical Example

```
const os = require('os');

console.log("Hostname:", os.hostname());

console.log("Platform:", os.platform());

console.log("Architecture:", os.arch());

console.log("CPU Cores:", os.cpus().length);

console.log("Free Memory (MB):",
Math.round(os.freemem() / 1024 / 1024));

console.log("Total Memory (MB):",
Math.round(os.totalmem() / 1024 / 1024));

console.log("User Info:", os.userInfo());

console.log("Home Directory:", os.homedir());
```

4.2.5 events Module

The events module provides the ability to create and handle custom events. It implements the Observer design pattern, where event listeners respond to emitted events.Central to Node.js architecture (used internally in HTTP, streams, etc.)Used for building loosely coupled and reactive components.

Everything in Node (like http, fs, process, stream) extends from EventEmitter.

Common Use Cases

- Logging
- Notification systems
- Message queues
- Custom application logic triggers

Practical Example

```
const EventEmitter = require('events');

// Step 1: Create custom emitter class

class MyEmitter extends EventEmitter {}

const emitter = new MyEmitter();


// Step 2: Register event listeners

emitter.on('start', () => console.log('Started
process'));

emitter.on('data', (value) => console.log(`Received:
${value}`));

emitter.on('end', () => console.log('Process ended'));

// Step 3: Emit events

emitter.emit('start');

emitter.emit('data', 'File uploaded');

emitter.emit('data', 'Image processed');

emitter.emit('end');
```

5) Express.js Framework

Express.js is a fast, minimalist web framework for Node.js that simplifies backend and API development. It provides a thin layer of fundamental web application features without obscuring the powerful core features of Node.js. Express is a part of MEAN stack, a full stack JavaScript solution used in building fast, robust, and maintainable production web applications. Express enables you to quickly create APIs and server-side applications using JavaScript.

5.1 Why Use Express.js?

Node.js by itself (via the http module) is powerful, but building robust web apps with just it becomes verbose and repetitive. Express solves this by offering:

Feature	Benefit
Minimalist Syntax	**Define routes and logic easily**
Rich Middleware Support	**Reusable functions to handle requests**
REST API-ready	**Ideal for creating JSON-based APIs**
Template Engine Support	**Supports ejs, pug, handlebars, etc.**
Easy Request/Response Handling	**Provides req and res objects with many helpers**
Strong Ecosystem	**Works well with MongoDB, Mongoose, Passport, etc.**

6. Setting up an Express server

This installs it locally and adds it to your package.json.

```
npm install express
```

Typical Express Project Structure

```
express-app/
├── app.js
├── package.json
├── node_modules/
└── routes/
    └── userRoutes.js   (optional)
```

6.1 Approach

- Use require('express') to import the Express module.
- Call express() to create an Express application instance.
- Define the port for the application, typically 3000.
- Set up a basic GET route with app.get('/', (req, res) => res.send('Hello World!')).
- Use app.listen method to listen to your desired PORT and to start the server.

Example: Implementation to setup the express application.

app.js

```
const express = require('express');

const app = express();

const PORT = 3000;

// Middleware to parse JSON

app.use(express.json());

// Root Route

app.get('/', (req, res) => {

  res.send('Welcome to Express.js!');

});

// About Route

app.get('/about', (req, res) => {

  res.send('This is the About Page.');

});

// Start Server

app.listen(PORT, () => {

  console.log(`Server running at http://localhost:${PORT}`);

});
```

Run the Server:

```
node app.js
```

Open browser:

- **http://localhost:3000/**
- **http://localhost:3000/about**

6.2 Setting GET request route on the root URL

- Use **app.get()** to configure the route with the path **'/'** and a callback function.
- The **callback function** receives **req (request)** and **res (response)** objects provided by Express.
- **req i**s the incoming request object containing client data, and **res** is the response object used to send data back to the client.
- Use **res.status()** to set the HTTP status code before sending the response.
- Use **res.send()** to send the response back to the client. You can send a string, object, array, or buffer. Other response methods include **res.json() for JSON** objects and **res.sendFile()** files.

6.3 Setting route to be accessed by users to send data with post requests.

- Before creating a route for receiving data, we are using an inbuilt middleware, **Middleware** is such a broad and more advanced topic so we are not going to discuss it here, just to understand a little bit you can think of this as a piece of code **that gets executed between the request-response cycles**.
- The **express.json() middleware** is used to parse the incoming request object as a JSON object. The **app. use() is** the syntax to use any middleware.
- After that, we have created a route on path '/' for post request.

7. Routing

Routing refers to how an application responds to client requests (like GET, POST, etc.) to specific **endpoints or URLs**. Express provides functions like **app.get(), app.post(), app.put(), app.delete()** to handle HTTP methods.

7.1 Why Routing Matters

Reason	Description
Entry Points	Maps HTTP requests to logic handlers
RESTful APIs	Supports full CRUD operations via different routes
Separation of Logic	Organize logic into modular route files
Performance	Direct path matching is fast and efficient

7.2 Practical:-

express-routing-app/

├── app.js

└── routes/

└── users.js

Step1 :- routes/users.js – Modular User Routes

```
const express = require('express');

const router = express.Router();

// Dummy user data

let users = [

  { id: 1, name: "Alice" },
```

```
  { id: 2, name: "Bob" },
  { id: 3, name: "Charlie" }
];

// GET /users - all users
router.get('/', (req, res) => {
  res.json(users);
});

// GET /users/:id - single user
router.get('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const user = users.find(u => u.id === id);
  if (user) res.json(user);
  else res.status(404).json({ message: 'User not found' });
});

// POST /users - add new user
router.post('/', (req, res) => {
  const newUser = {
    id: users.length + 1,
    name: req.body.name
  };
  users.push(newUser);
  res.status(201).json(newUser);
```

```
});

// PUT /users/:id - replace user
router.put('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = users.findIndex(u => u.id === id);
  if (index !== -1) {
    users[index] = { id, name: req.body.name };
    res.json(users[index]);
  } else {
    res.status(404).json({ message: 'User not found' });
  }
});

// PATCH /users/:id - update user partially
router.patch('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const user = users.find(u => u.id === id);
  if (user) {
    user.name = req.body.name || user.name;
    res.json(user);
  } else {
    res.status(404).json({ message: 'User not found' });
  }
});
```

```
// DELETE /users/:id - delete user
router.delete('/:id', (req, res) => {
  const id = parseInt(req.params.id);
  const index = users.findIndex(u => u.id === id);
  if (index !== -1) {
    const removed = users.splice(index, 1);
    res.json(removed[0]);
  } else {
    res.status(404).json({ message: 'User not found' });
  }
});

// GET /users/search?name=Bob
router.get('/search/name', (req, res) => {
  const query = req.query.name?.toLowerCase();
  const result = users.filter(u =>
u.name.toLowerCase().includes(query));
  res.json(result);
});

module.exports = router;
```

Step 2 :- app.js – Main Server File

```
const express = require('express');
const app = express();
const userRoutes = require('./routes/users');
```

```
// Middleware to parse JSON
app.use(express.json());
// Routes
app.use('/users', userRoutes);
// Fallback route
app.use((req, res) => {
  res.status(404).json({ message: 'Endpoint not found' });
});


// Start server
app.listen(3000, () => {
  console.log('Server is running on http://localhost:3000');
});
```

Step 3:- Run the file on terminal by using the command:

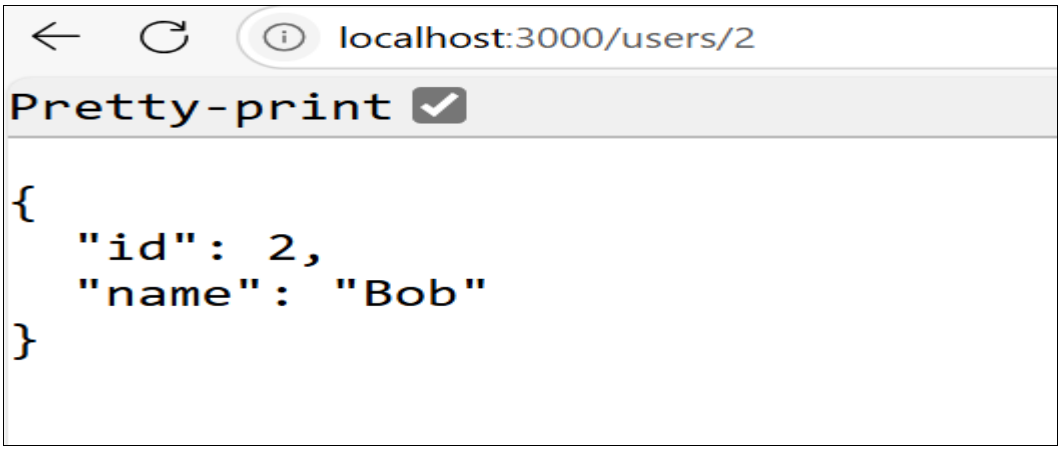
```
Node app.js
```

Step 4:- GET /users

In browser url type localhost:3000/users

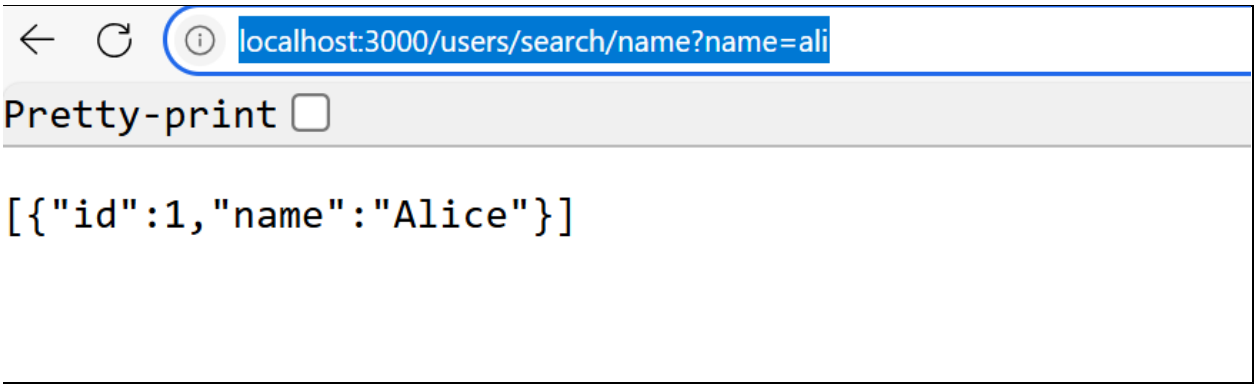
Step 5:- GET /users/2

In browser url type localhost:3000/users/2



Step 6:- GET /users/search/name?name=ali

In browser url type:-localhost:3000/users/search/name?name=ali



Step 7:- now in order to check other routes like post, put, delete, patch open your postman tool for testing purpose & select appropriate request method.

POST /users

Body (JSON): { "name": "Daisy" }

Response: { "id": 4, "name": "Daisy" }

Step 8:- PUT /users/2

Body (JSON): { "name": "Robert" }

Response: { "id": 2, "name": "Robert" }

Step 9:- PATCH /users/3

Body (JSON): { "name": "Charles" }

Response: { "id": 3, "name": "Charles" }

Step 10:- DELETE /users/1

Response: { "id": 1, "name": "Alice" }

8. Middleware

Middleware are functions that have access to **req, res, and next**. They're executed in order. Middleware functions can **perform the following tasks:**

- Middleware simplifies software development
- It controls connections between application components
- No doubt, it is a cost-effective tool for managing multi-cloud resources.
- It plays a crucial role in load balancing, concurrent processing, and transaction management.
- With middleware, you can make secured access to back-end resources such as storage, databases, etc.
- It supports authentication functionalities on a greater scale
- You can enhance rendering performance for client-side significantly
- By using middleware, you can easily set HTTP headers

With middleware Node.JS, we can do a multitude of things. we can run any codes with middleware functions. Also, we can make changes in response and request objects. We can end the request and response cycle in Node.JS execution. With middleware, you don’t need to make custom integrations every time.

8.1 Types of Node.JS Middleware :-

1. **Application-level Middleware**
    - Bound to an instance of express()
    - Used across the entire application

```
app.use((req, res, next) => {
  console.log('Time:', Date.now());
  next();
});
```

2. **Router-level Middleware**
    - Similar to application-level, but bound to an instance of express.Router()

```
const router = express.Router();
router.use((req, res, next) => {
  console.log('Router middleware');
  next();
});
```

3. **Built-in Middleware**
    - Comes with Express.js (v4.16+)
    - Examples: express.json(), express.urlencoded(), express.static()

```
app.use(express.json());
```

4. **Error-handling Middleware**
    - Must have 4 parameters: (err, req, res, next)
    - Used to catch and handle errors in the middleware chain

```
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).send('Something broke!');
});
```

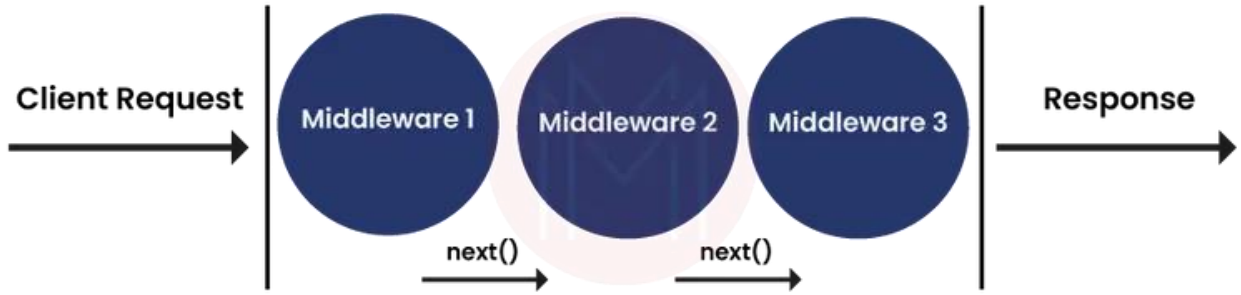
5. **Third-party Middleware**
    - Installed via **npm**
    - Examples: **morgan, body-parser, cors**

```
const morgan = require('morgan');

app.use(morgan('dev'));
```

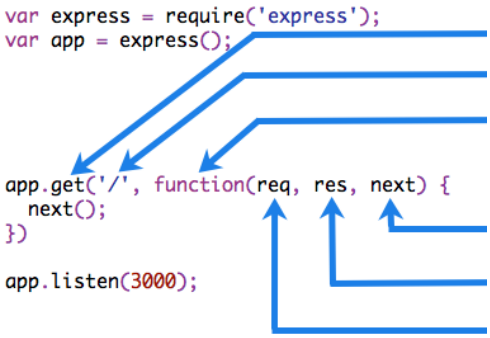
Note:- Middleware **runs in the order** it's registered, so place general-purpose middleware (like logging**) before route handlers, and error-handling middleware at the end.**

8.2 What is the Next () Function?

The next () function plays a vital role in applications' request and response cycle. It is a middleware function that runs the next middleware function once it is invoked. In other words, the Next function is invoked if the current middleware function doesn't end the request and response cycle. It is essential to note that no middleware function should be hanging in the queue. Starting with Express 5, middleware functions that return a Promise will call next(value) when they reject or throw an error. next will be called with either the rejected value or the thrown Error.



The image below is the middleware function where you can find its different components.



HTTP method for which the middleware function applies.

Path (route) for which the middleware function applies.

The middleware function.

Callback argument to the middleware function, called "next" by convention.

HTTP response argument to the middleware function, called "res" by convention.

HTTP request argument to the middleware function, called "req" by convention.

8.3 Built-in middleware:

- **Static:** They are functions that act as static assets to applications. HTML files and images are a few examples of static assets.
- **JSON:** This function processes incoming requests along with the JSON payloads.
- **Express.URL-encoded:** This function processes incoming requests along with URL-encoded payloads.

8.4 Complete Practical:- Prac.js

```
const express = require('express');
const app = express();


// Application-level middleware
app.use((req, res, next) => {
  console.log('Request received:', req.method,
req.url);
  next();
```

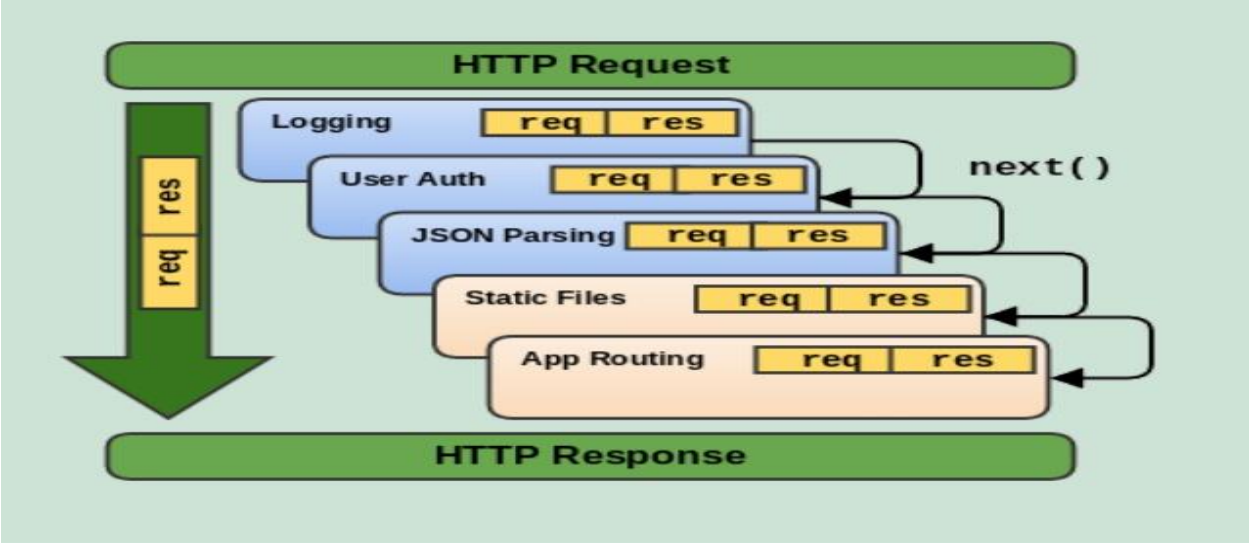
```
});

// Built-in middleware

app.use(express.json());


// Route handler

app.get('/', (req, res) => {

  res.send('Hello, World!');

});


// Error-handling middleware

app.use((err, req, res, next) => {

  res.status(500).send('Internal Server Error!');

});


app.listen(3000, () => console.log('Server running on port 3000'));
```

9. Handling Request & Response Objects

In Express.js, every route you define interacts with two main objects:

Object	Purpose
req	Represents the **incoming request** from the client
res	Represents the **outgoing response** to the client

These two objects are passed automatically into each route's callback function.



9.1 Common Use Cases for req and res

Feature	Method/Property	Description
Route Parameters	**req.params**	Extract dynamic URL parts (e.g., /user/:id)
Query Strings	**req.query**	Extract data from ?key=value in URL
Request Body	**req.body**	Extract POST/PUT body data (requires express.json())
Headers	**req.headers, res.set()**	Read/write headers
JSON Response	**res.json()**	Send response in JSON
Plain Text Response	**res.send()** **or** **res.write()**	Send response in text format
Set HTTP Status	**res.status(code)**	Set response status code
Redirect	**res.redirect('/path')**	Redirect client to another URL

9.2 Express vs. Node http Module

Feature	Native http	Express.js
Syntax	Verbose	Clean and declarative
Routing	Manual path matching	Built-in
Body Parsing	Manual	Built-in JSON support
Middleware	Hard to implement	Native to Express
Ecosystem	Low	Huge (middleware, plugins)

9.3 Real-World Use Cases

Use Case	Description
RESTful APIs	Expose backend endpoints for client apps
Full-stack Web Apps	Combine with EJS/React frontends
Authentication Systems	Use with Passport.js or JWT
CRUD Applications	With MongoDB or SQL
Microservices	Lightweight and easy to scale

10.Database Integration in Node.js

Database integration in a Node.js application means connecting your app to a persistent storage system (like MongoDB, MySQL, or PostgreSQL) so that data can be:

- Stored (Create)
- Retrieved (Read)
- Modified (Update)
- Removed (Delete)

Node.js can integrate with both SQL (Relational) and NoSQL (Document-based) databases using libraries or ODM/ORM tools.

10.1 Overview of Database

A database is a structured collection of data that is organized, managed, and accessed electronically. It is designed to store, retrieve, and manipulate large amounts of data efficiently and securely. Databases are used in various applications and industries to store and manage data, ranging from simple contact lists to complex enterprise systems.

Here's an overview of key components and concepts related to databases:

- **Data:** Databases store data, which can be organized into tables, records, and fields. Data can be numeric, text, dates, images, or any other type of information.

- **Database Management System (DBMS):** The DBMS is software that allows users to interact with the database. It provides tools and functions for creating, modifying, querying, and managing the database. Popular DBMSs include Oracle, MySQL, Microsoft SQL Server, PostgreSQL, and MongoDB.

- **Data Models:** Data models define the structure and relationships of the data in a database. The two most common data models are the relational model and the NoSQL model. The relational model uses tables with predefined

relationships, while NoSQL databases use flexible data structures like key-value pairs, documents, or graphs.

- **Schema:** The schema defines the logical structure of the database. It includes the tables, fields, relationships, and constraints. A schema acts as a blueprint for the database, ensuring data consistency and integrity.

- **Query Language:** Databases use query languages to retrieve and manipulate data. Structured Query Language (SQL) is the most widely used language for relational databases. NoSQL databases may have their own query languages or provide APIs for data access.

- **CRUD Operations:** CRUD stands for Create, Read, Update, and Delete, which are the fundamental operations performed on database records. Users can create new records, read existing records, update the data, and delete unwanted records.

- **Indexing:** Indexing improves the performance of database queries by creating data structures that allow for efficient searching and retrieval. Indexes are created on specific columns to speed up data retrieval operations.

- **Transactions:** A transaction is a unit of work performed on the database. It represents a sequence of operations that should be treated as a single, atomic unit. Transactions ensure data consistency and integrity by enforcing the concept of "all-or-nothing" execution.

- **Security:** Database security is essential to protect sensitive data. Access control mechanisms, authentication, and encryption techniques are used to ensure that only authorized users can access and modify the database.

- **Backup and Recovery:** Regular backups are taken to protect against data loss due to hardware failures, human errors, or disasters. Backup strategies include full backups, incremental backups, and off-site storage. Recovery

procedures are in place to restore the database to a consistent state in case of data loss or corruption.

Databases play a crucial role in modern systems and applications by providing a structured and efficient way to store, manage, and retrieve data. They enable businesses to make informed decisions, improve efficiency, and maintain data integrity and security.

10.2 Database Management System (DBMS)

A Database Management System (DBMS) is software that allows users to efficiently store, manage, retrieve, and manipulate large amounts of data. It serves as an intermediary between the users and the database, providing an interface to interact with the data stored in the database.

Here are some key features and functions of a DBMS:

- **Data Organization:** DBMS provides mechanisms for organizing and structuring data in a logical manner. It allows users to define data schemas, tables, and relationships between tables to ensure data integrity and consistency.

- **Data Manipulation:** DBMS enables users to perform various operations on the data, such as inserting new data, updating existing data, deleting data, and querying data to retrieve specific information. This is typically done using query languages like SQL (Structured Query Language).

- **Data Security:** DBMS provides security measures to protect data from unauthorized access, ensuring that only authorized users can access and modify the database. It includes features like user authentication, access control, and encryption to safeguard sensitive data.

- **Data Integrity:** DBMS enforces data integrity by applying rules and constraints on the data. It ensures that the data stored in the database is accurate, consistent, and valid. Constraints such as primary keys, foreign

keys, unique constraints, and check constraints are used to maintain data integrity.

- **Data Concurrency:** DBMS manages concurrent access to the database by multiple users or applications. It provides mechanisms to handle simultaneous data modifications and ensures that changes made by one user do not conflict with changes made by others. This includes features like locking, transaction management, and concurrency control.

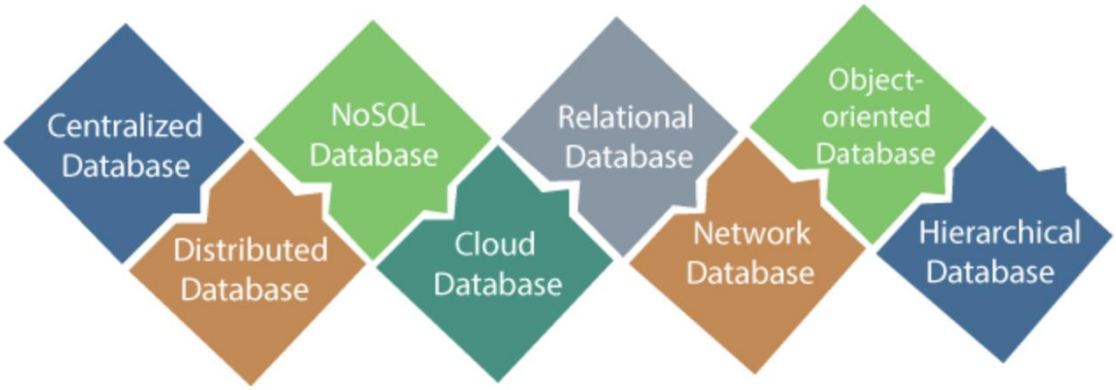
- **Data Backup and Recovery:** DBMS allows users to create backups of the database to prevent data loss in case of hardware failures, software errors, or disasters. It provides mechanisms for restoring the database to a previous state using backup files.

- **Data Scalability and Performance:** DBMS is designed to handle large volumes of data and provide efficient access and retrieval mechanisms. It optimizes query execution, manages indexes, and employs various performance tuning techniques to ensure efficient data processing.

- **Data Independence:** DBMS provides data independence by separating the logical representation of data from its physical storage. This allows users and applications to interact with the database without worrying about the underlying storage details, making it easier to modify the database structure or migrate to a different storage system.

DBMS plays a critical role in managing and maintaining data in organizations. It provides a centralized and controlled environment for storing and accessing data, ensuring data consistency, security, and efficiency. By utilizing a DBMS, organizations can effectively handle and leverage their data assets for decision-making, analysis, and business operations.

10.3 Types of Databases



There are several types of databases, each designed to handle specific data storage and management requirements. Here are some common types of databases:

- **Relational Database:** Relational databases organize data into tables with rows and columns, where each table represents an entity or relationship. They use Structured Query Language (SQL) for data manipulation and have a predefined schema that enforces data integrity. Examples of relational databases include Oracle Database, MySQL, and Microsoft SQL Server.

- **NoSQL Database:** NoSQL databases, or "Not Only SQL" databases, provide a flexible data model that allows for the storage and retrieval of unstructured, semi-structured, and structured data. They are designed to handle large volumes of data and offer scalability and high performance. NoSQL databases include document databases (e.g., MongoDB), key-value stores (e.g., Redis), columnar databases (e.g., Apache Cassandra), and graph databases (e.g., Neo4j).

- **Object-Oriented Database:** Object-oriented databases store data in the form of objects, which encapsulate data and behavior. They support complex data types and inheritance, enabling the storage of rich, structured data.

Object-oriented databases are often used in applications where the data model closely matches the object-oriented programming paradigm.

- **Hierarchical Database:** Hierarchical databases organize data in a tree-like structure, with parent-child relationships between data elements. Each child can have only one parent, representing a strict one-to-many relationship. Hierarchical databases are less commonly used today but can still be found in legacy systems or specialized applications.

- **Network Database:** Network databases extend the hierarchical model by allowing many-to-many relationships between entities. They use pointers to navigate between records and establish complex relationships. Network databases are also less prevalent today but were used in some legacy systems.

- **Time-Series Database:** Time-series databases specialize in storing and analyzing time-stamped data, such as sensor data, financial market data, or log data. They optimize storage and retrieval of time-series data, enabling efficient time-based queries and analysis.

- **Spatial Database:** Spatial databases are designed to store and process spatial or geographic data, such as maps, satellite imagery, or location data. They include specialized data types and indexing techniques to support spatial operations like distance calculations, polygon intersections, and spatial queries.

- **In-Memory Database:** In-memory databases store data primarily in the system's main memory (RAM) rather than on disk. They provide ultra-fast data access and processing, making them suitable for applications that require high-performance data operations.

These are some of the commonly recognized types of databases. It's important to note that there can be overlaps and hybrid approaches, as different databases can incorporate features from multiple types. The choice of database type depends on the specific requirements, data model, scalability needs, and performance considerations of the application or system being developed.

10.4 Key Differences Between SQL & NoSQL

Feature	SQL (Relational)	NoSQL (Non-Relational)
Schema	Fixed (tables, columns, types)	Flexible, dynamic
Data Format	Tables with rows & columns	JSON, BSON, key-value, etc.
Primary Key	Required	Optional or autogenerated
Scaling	Vertical (scale-up)	Horizontal (scale-out easily)
Relationships	Strong, foreign keys, joins	Weak, typically avoided
Use Cases	Structured, transactional data	Unstructured, large, real-time
Complexity	More structured setup, SQL queries	Quick development, JS-friendly

10.5 SQL Databases – When to Use?

Use SQL when:

- You need **strict data consistency**
- Your data is **structured** (like invoices, orders, user accounts)
- You rely heavily on **relations** (foreign keys)
- You need **ACID** (Atomicity, Consistency, Isolation, Durability)

Use Case Examples:

- Banking systems, HR systems
- E-commerce checkout systems, Inventory tracking

10.6 NoSQL Databases – When to Use?

Use NoSQL when:

- Your data structure changes often
- You're storing **JSON-like documents**
- You need **scalability** and **speed**
- You need to handle **large volumes of data**

Use Case Examples:

- Real-time chat apps, Product catalogs
- IoT telemetry data
- User-generated content, Dynamic user profiles

10.7 How to Connect Node.js to a Database

Step 1:- Install the database client package **(e.g., mongoose, mysql2)**

npm install mongoose

Step 2:- **Create a connection to the database**

Step 3:- **Query/insert/update/delete data using the API**

Step 4:- **Handle errors and close connections**

11. Introduction to MongoDB

MongoDB is a document database which is often referred to as a non-relational database. This does not mean that relational data cannot be stored in document databases. It means that relational data is stored differently. A better way to refer to it is as a non-tabular database.

MongoDB stores data in flexible documents. Instead of having multiple tables you can simply keep all of your related data together. This makes reading your data very fast.

11.1 Key Characteristics

Feature	Description
Document-Oriented	Stores data in BSON (like JSON objects)
Schema-less	Collections don’t enforce strict schemas
Scalable	Supports horizontal scaling (sharding)
Fast and Efficient	Ideal for high-throughput apps
Real-time	Great for apps needing instant reads/writes
ACID Transactions	Supported from MongoDB 4.0+

11.2 Example Document in MongoDB

```
{
  "_id": "abc123",
  "name": "Alice",
  "email": "alice@example.com",
  "isActive": true,
```

```
  "tags": ["student", "gamer"],
  "address": {
    "city": "Delhi",
    "pincode": 110001
  }
}
```

Documents can have nested objects, arrays, and different structures even within the same collection.

11.3 Why Use MongoDB in Node.js?

Reason	Description
JSON-Like Format	Pairs naturally with JavaScript & Node
Dynamic Schema	Easily update data structure as needed
High Performance	Efficient reads/writes for APIs
Rich Query Language	Supports complex filtering, sorting
Mongoose ODM Support	Use mongoose to enforce schemas & models

11.4 Difference Between MongoDB Atlas and MongoDB Compass

Here is a detailed comparison of MongoDB Atlas and MongoDB Compass

Feature	MongoDB Compass	MongoDB Atlas
Purpose	A GUI tool for interacting with MongoDB databases locally.	A fully-managed cloud database service for MongoDB.

Feature	MongoDB Compass	MongoDB Atlas
Target Audience	Developers and database administrators working on local database management.	Organizations and developers looking for scalable, cloud-based database solutions.
Deployment	Installed on the local machine.	Deployed in the cloud on platforms like AWS, Azure, and Google Cloud.
Functionality	Simplifies data exploration, querying, and administrative tasks through a visual interface.	Automates database scaling, backups, security, and global distribution for seamless cloud operations.
Scalability	Not designed for scalability; limited to local database management.	Highly scalable; handles large datasets with automated scaling options.
Query Building	Includes a visual query builder to construct complex queries easily.	No built-in query builder; accessed through APIs or drivers for database interactions.
Real-Time Stats	Provides limited real-time server statistics.	Offers detailed real-time server statistics and performance monitoring tools.
Security	No significant built-in security features.	Provides robust security, including encryption, access control, and network isolation.

11.5 Key Use Cases

MongoDB Compass

- Local database management and exploration.
- Ideal for developers who need a visual tool to query, analyze, and manage MongoDB data on their machines.
- Suitable for small-scale projects that don't require scalability or global distribution.
- Download:- https://www.mongodb.com/products/compass

MongoDB Atlas

- Cloud-based applications requiring high scalability and availability.
- Enterprises handling large datasets with frequent backups and disaster recovery needs.
- Projects requiring global distribution for low-latency data access.
- Organizations prioritizing database security and automated management.
- You get: Free hosted clusters (up to 512 MB)

12. Setting up MongoDB with Mongoose

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js.

It helps structure MongoDB data using schemas and models, making it easy to:

- Validate data
- Use custom methods
- Work with queries using an intuitive API

12.1 Setup mongoose

Step 1:- **npm install mongoose**

Step 2:- Connect to MongoDB

db.js

```
const mongoose = require('mongoose');

mongoose.connect('mongodb://localhost:27017/myapp')

  .then(() => console.log('Connected to MongoDB'))

  .catch((err) => console.error('Connection failed', err));
```

You can replace localhost with your MongoDB Atlas URI for cloud-hosted DB.

Step 3:- Define a Mongoose Schema & Model

models/User.js

```
const mongoose = require('mongoose');


// Schema defines shape of data

const userSchema = new mongoose.Schema({

  name: { type: String, required: true },

  email: { type: String, unique: true, required: true },

  isActive: { type: Boolean, default: true },

  createdAt: { type: Date, default: Date.now }

});

// Create model

const User = mongoose.model('User', userSchema);

module.exports = User;
```

Step 4:- Using the Model in Routes or Logic

app.js

```
const express = require('express');

const app = express();

const mongoose = require('mongoose');

const User = require('./models/User');

app.use(express.json());

// Connect DB

mongoose.connect('mongodb://localhost:27017/myapp')

  .then(() => console.log('MongoDB connected'))

  .catch(err => console.error('MongoDB connection error:', err));


// Create new user

app.post('/users', async (req, res) => {

  try {

    const user = new User(req.body);

    const result = await user.save();

    res.status(201).json(result);

  } catch (err) {

    res.status(400).json({ error: err.message });

  }

});

// Get all users

app.get('/users', async (req, res) => {
```

```
  const users = await User.find();
  res.json(users);
});
app.listen(3000, () => {
console.log('Server running on
http://localhost:3000');
});
```

Step 5:- Run using command node app.js

Step 6:- Test (POST request via Postman)

Request:- POST /users

```
{
  "name": "Preenu",
  "email": "preenu@example.com"
}
```

Response:

```
{
  "_id": "662f...a8",
  "name": "Preenu",
  "email": "preenu@example.com",
  "isActive": true,
  "createdAt": "2025-06-11T11:30:00.000Z"
}
```

12.2 Mongoose Schema Options

Option	Usage Example	Description
type	type: String	Data type
required	required: true	Field must exist
default	default: Date.now	Default value
unique	unique: true	Must be unique in collection

13) CRUD Operations with MongoDB

CRUD allows a complete data lifecycle within your application – from insertion to deletion. CRUD is a foundational concept in application development and stands for:

Operation	Action in App	HTTP Method
Create	Add new records (e.g., new user)	**POST**
Read	Retrieve data (list or details)	**GET**
Update	Modify existing data	**PUT / PATCH**
Delete	Remove records from the database	**DELETE**

13.1 Why Use CRUD with Mongoose?

Mongoose is an ODM that allows us to:

- Define clear data models (schemas)
- Interact with MongoDB using JavaScript methods

- Validate and transform data with ease
- Perform async operations using await / promises

13.2 Mongoose CRUD APIs

Action	Method/Function	Description
Create	**Model.create(), new Model() + .save()**	Adds new document
Read (All)	**Model.find()**	Returns all matching documents
Read (By ID)	**Model.findById(id)**	Fetch document by _id
Update	**Model.findByIdAndUpdate()**	Modify and return updated doc
Delete	**Model.findByIdAndDelete()**	Remove document by _id

13.3 Mongoose Schema Design

Mongoose uses schemas to enforce data structure:

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, unique: true, required: true },
  isActive: { type: Boolean, default: true },
  createdAt: { type: Date, default: Date.now }
});
```

Benefits:

- Input validation
- Enforces consistency,Custom methods and hooks

13.4 Handling CRUD Operations

1. CREATE – POST /users

Purpose: Add a new user to the database.

Flow:

- Client sends JSON in the request body.
- Mongoose validates against schema.
- A new document is saved to the users collection.

```
const user = new User(req.body);

await user.save();
```

Validation checks include:

- Required fields
- Unique email
- Default values (e.g., createdAt, isActive)

2. READ (All) – GET /users

Purpose: Fetch all users.

Flow: Uses User.find() to return all documents.You can add pagination, filtering, or sorting later.

```
const users = await User.find();
```

3. READ (By ID) – GET /users/:id

Purpose: Fetch a specific user by _id.

```
const user = await User.findById(req.params.id);
```

- Handles errors if ID is invalid or user doesn't exist.
- _id is a unique MongoDB ObjectId.

4. UPDATE – PATCH /users/:id

Purpose: Modify one or more fields of a document.

```
const updated = await User.findByIdAndUpdate(id,
updates, {
  new: true,
  runValidators: true
});
```

- **new: true** returns **updated document**
- **runValidators: true** ensures the **update follows schema rules**
- Use **PUT to replace the entire object**; **PATCH** for **partial updates**.

5. DELETE – DELETE /users/:id

Purpose: Remove a user from the collection.

- Removes the document permanently
- You can also use soft delete (**e.g., isDeleted: true**) for reversible logic

```
const deleted = await
User.findByIdAndDelete(req.params.id);
```

13.5 Error Handling in CRUD

- Invalid IDs → CastError

- Missing fields → ValidationError
- Duplicate email → MongoDB index error
- Missing document → return 404

You should **always wrap CRUD operations in try...catch blocks** or use **Express error middleware.**

13.6 Security Notes

Tip	Why
Validate user inputs	Prevent malformed/malicious data
Sanitize query params	Avoid injection or bypasses
Use try/catch blocks	Avoid app crashes on DB errors
Never trust req.body blindly	Always validate/sanitize input

13.7 Mini Project: User Management System (CRUD API)

Tech Stack

- Backend: Node.js
- Framework: Express.js
- Database: MongoDB
- ODM: Mongoose

Folder Structure

user-crud-api/

├— models/

```
|   └── User.js
├── routes/
|   └── userRoutes.js
├── db.js
├── app.js
├── .env
├── package.json
```

Step 1. Initialize Project

```
mkdir user-crud-api && cd user-crud-api
npm init -y
npm install express mongoose dotenv
```

Step 2. .env (MongoDB URI)

```
MONGO_URI=mongodb://localhost:27017/userdb
PORT=3000
```

(Use MongoDB Atlas URL if needed)

Step 3. models/User.js

```
const mongoose = require('mongoose');
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  age: Number,
  isActive: { type: Boolean, default: true },
```

```
  createdAt: { type: Date, default: Date.now }
});
module.exports = mongoose.model('User',
userSchema);
```

Step 4. routes/userRoutes.js

```
const express = require('express');
const router = express.Router();
const User = require('../models/User');
// Create
router.post('/', async (req, res) => {
  try {
    const user = await User.create(req.body);
    res.status(201).json(user);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
// Read All
router.get('/', async (req, res) => {
  const users = await User.find();
  res.json(users);
});
// Read One
router.get('/:id', async (req, res) => {
  try {
```

```
    const user = await User.findById(req.params.id);
    if (!user) return res.status(404).json({ message:
'User not found' });
    res.json(user);
  } catch {
    res.status(400).json({ message: 'Invalid ID' });
  }
});
// Update
router.patch('/:id', async (req, res) => {
  try {
    const updated = await
User.findByIdAndUpdate(req.params.id, req.body, {
      new: true, runValidators: true
    });
    if (!updated) return res.status(404).json({ message:
'User not found' });
    res.json(updated);
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});
// Delete
router.delete('/:id', async (req, res) => {
  try {
    const deleted = await
User.findByIdAndDelete(req.params.id);
```

```
    if (!deleted) return res.status(404).json({ message:
'User not found' });
    res.json({ message: 'User deleted', user: deleted });
  } catch {
    res.status(400).json({ message: 'Invalid ID' });
  }
});


module.exports = router;
```

Step 5. db.js

```
const mongoose = require('mongoose');
const dotenv = require('dotenv');
dotenv.config();
const connectDB = async () => {
  try {
    await
mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected');
  } catch (err) {
    console.error('DB connection error:', err);
    process.exit(1);
  }
};
module.exports = connectDB;
```

Step 6. app.js

```
const express = require('express');
const dotenv = require('dotenv');
const connectDB = require('./db');
const userRoutes = require('./routes/userRoutes');
dotenv.config();
const app = express();
// Middleware
app.use(express.json());

// Routes
app.use('/api/users', userRoutes);

// Server
connectDB();
const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Server running on http://localhost:${PORT}`));
```

13.8 API Test Summary (via Postman)

Method	Endpoint	Description
POST	/api/users	Create a new user
GET	/api/users	Get all users
GET	/api/users/:id	Get user by ID
PATCH	/api/users/:id	Update user details
DELETE	/api/users/:id	Delete user

13.9 Example POST Body

```
{
  "name": "Preenu Mittan",
  "email": "preenu@example.com",
  "age": 26
}
```

14. Authentication and Authorization

14.1 Authentication

Authentication is the act of **validating that users** are **whom they claim to be**. This is the first step in any security process. Complete an authentication process with:

- **Passwords.** Usernames and passwords are the most common authentication factors. If a user enters the correct data, the system assumes the identity is valid and grants access.
- **One-time pins:-** Grant access for only one session or transaction.
- **Authentication apps**. Generate security codes via an outside party that grants access.
- **Biometrics:-** A user presents a fingerprint or eye scan to gain access to the system.

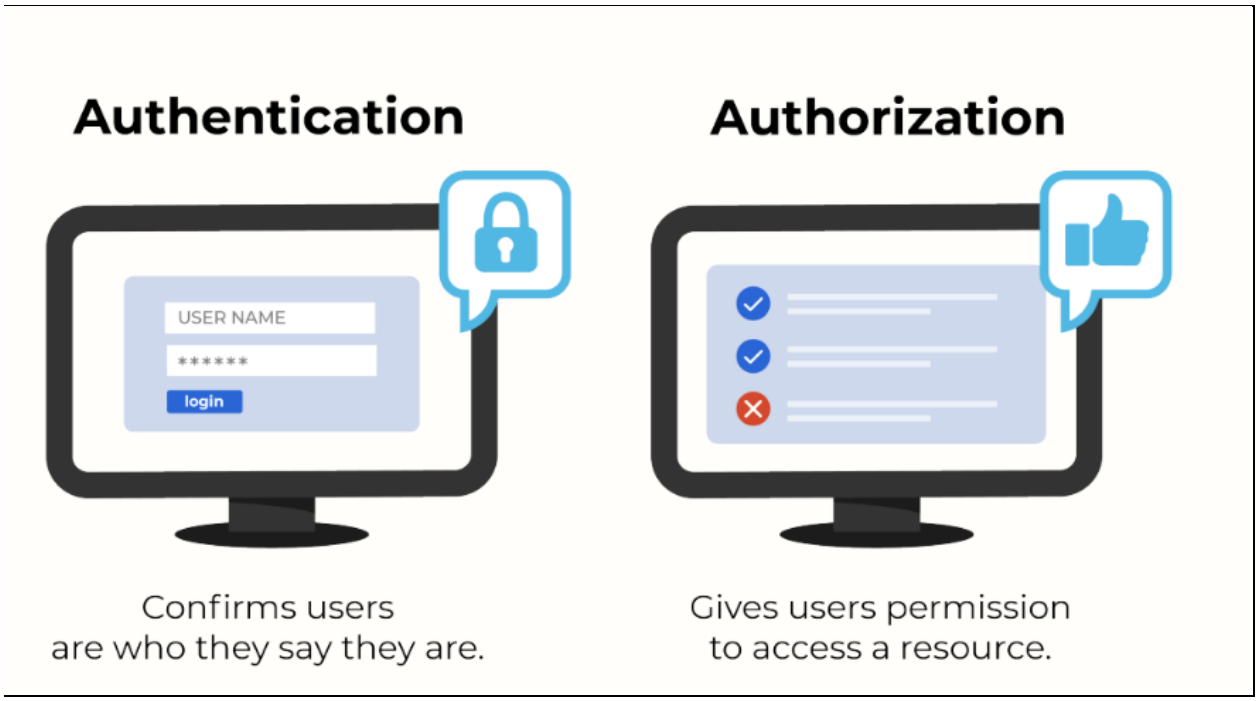
In some instances, systems require the successful verification of more than one factor before granting access. This multi-factor authentication (MFA) requirement is often deployed to increase security beyond what passwords alone can provide.

14.2 Authorization?

Authorization in system security is **the process of giving** the **user permission** to **access a specific resource or function**. This term is often used interchangeably with access control or client privilege.

Giving someone permission to download a particular file on a server or providing individual users with administrative access to an application are good examples of authorization.

In secure environments, authorization must always follow authentication. Users should first prove that their identities are genuine before an organization's administrators grant them access to the requested resources.



14.3 Authentication Types in Node.js Apps

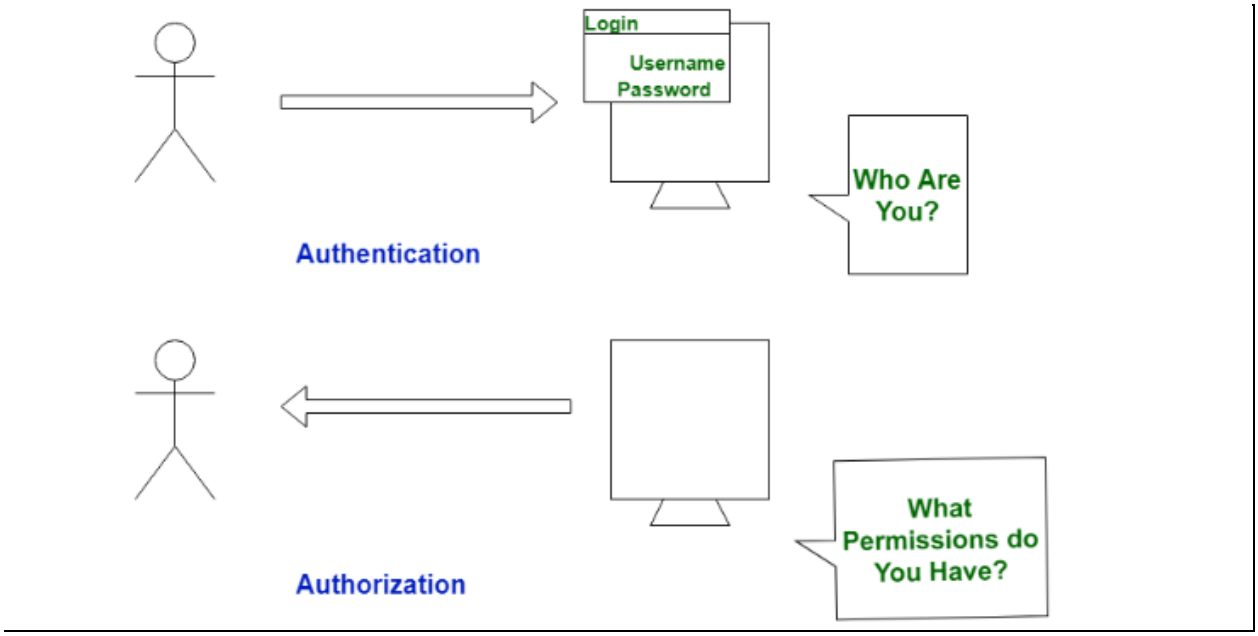
Type	Description	Tools Used
Local Auth	Username/email + password login	bcrypt, JWT, sessions

Type	Description	Tools Used
Token-based	User receives token after login	JWT (JSON Web Tokens)
OAuth	Third-party login (Google, GitHub, etc.)	Passport.js strategies
Session-based	Server maintains user sessions	express-session, cookies

14.4 Authorization in Practice

You use middleware to:

- Check if the user has a valid token
- Extract the user role
- Grant or deny access to specific routes



15)Understanding JWT (JSON Web Tokens)

JWT (JSON Web Token) is an open standard (RFC 7519) for securely transmitting information between parties as a JSON object.In Node.js apps, JWTs are commonly used for authentication — allowing users to log in and then access protected resources without storing sessions on the server.

15.1 Structure of JWT token

- Signature (to verify authenticity)
- Payload (user info, role, etc.)
- Header (algorithm, type)

15.2 Why Use JWT in Node.js?

Feature	Description
Stateless	No sessions; the token contains all info
Secure	Tamper-proof with HMAC SHA encryption
Expiry	Can auto-expire using `expiresIn`
Portable	Can be used across domains & mobile apps

15.3 How JWT Authentication Works

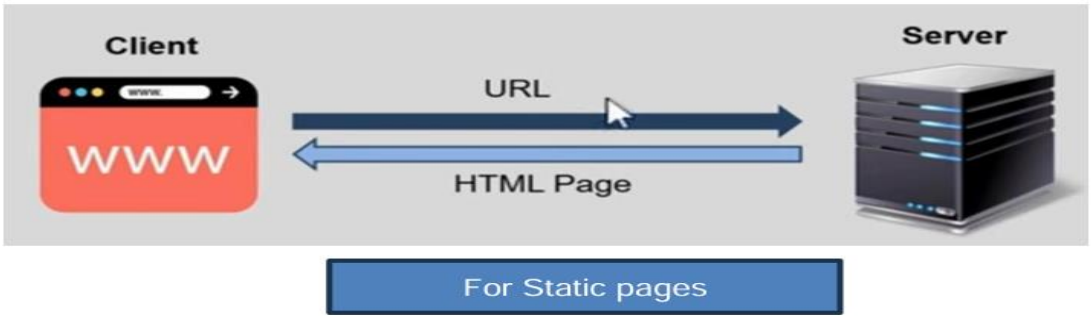
Step-by-Step:

1. **User logs in** with email + password

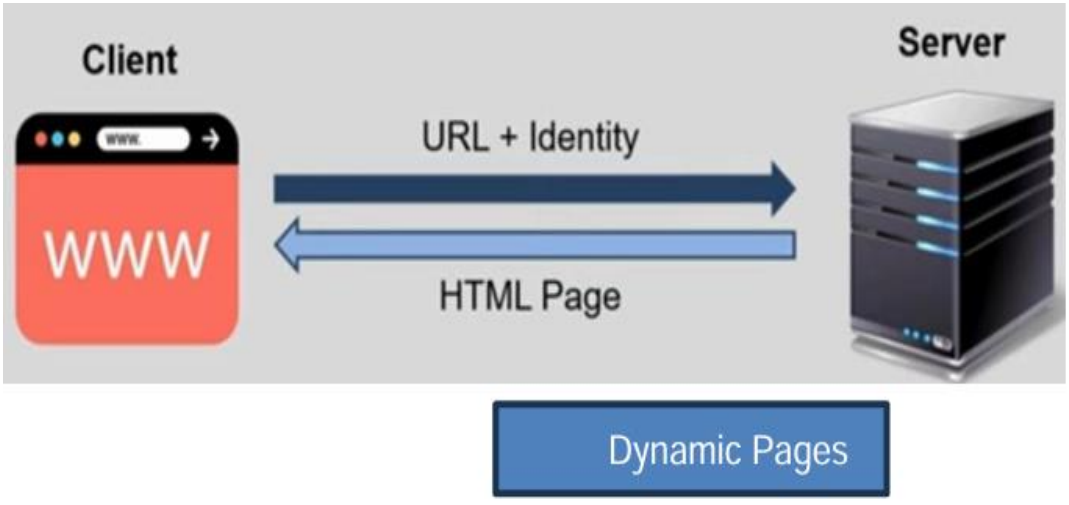
2. Server **verifies credentials**
3. If valid, server **creates a JWT** and sends it back
4. Client **stores the token** (localStorage or cookies)
5. For future requests, client sends token via Authorization header
6. Server **verifies token** and gives access to protected routes

15.4 HTTP: A Stateless Protocol

HTTP is a stateless protocol, meaning each request from a client (browser) to a server is independent and does not retain user state.



Clients send requests to servers to retrieve or submit data. - Since HTTP does not maintain session state, additional mechanisms like cookies, sessions, or tokens are required for persistent interactions.



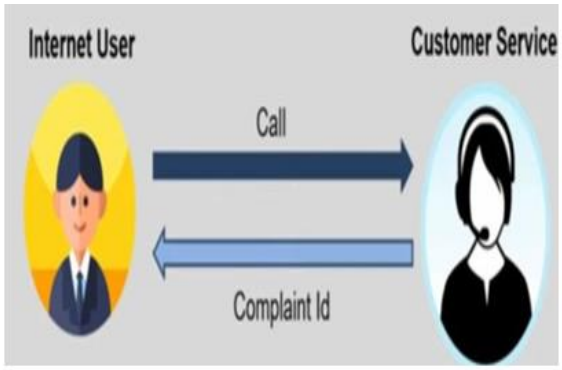
So in order to save info on server we use two things

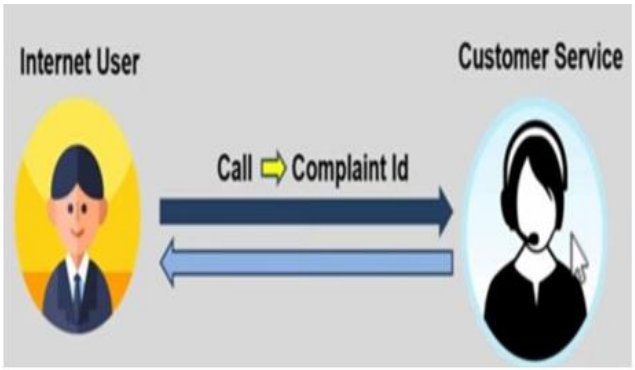
1.session

2. JWT

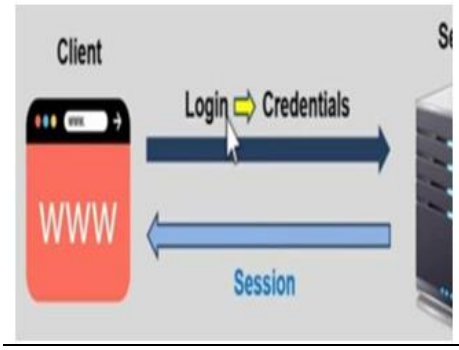
15.5 Understanding Sessions & How Sessions Work

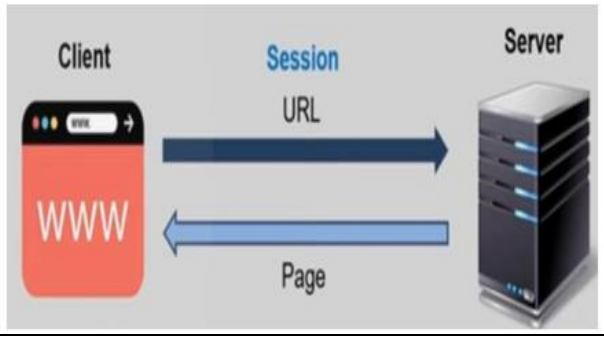
When a user logs into a website, a unique session ID is generated and stored on the server. his session ID is sent to the client as a cookie. On subsequent requests, the client includes the session ID, allowing the server to recognize the user.



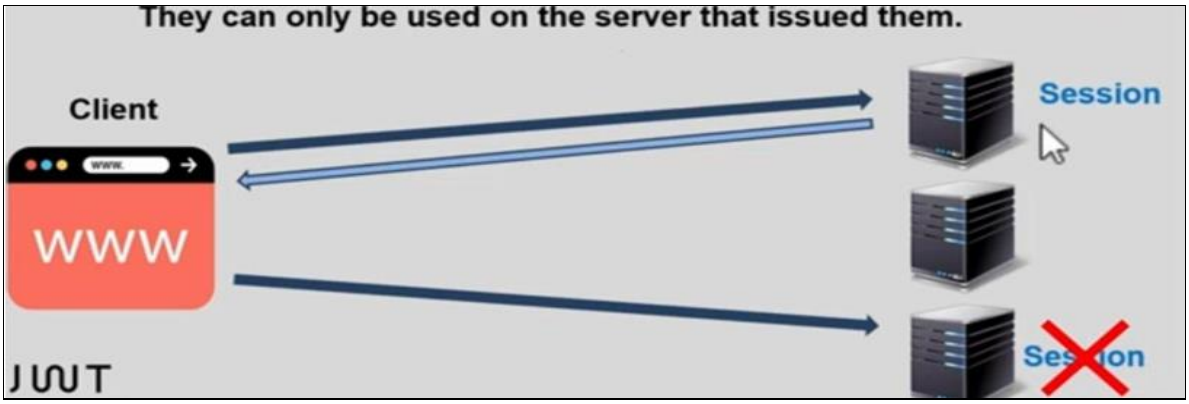


Session in Web Apps:-





Drawbacks of Sessions:- They are stored on the server, leading to scalability challenges. - Sessions are typically limited to the server that created them. - If the session expires or is lost, the user needs to log in again.



15.6. JWT (Json web Tokens)

A JWT is a compact, self-contained token used for authentication and authorization.

It consists of three parts: Header, Payload, and Signature. - Also knowns as Value tokens & sessions known as Ref tokens.

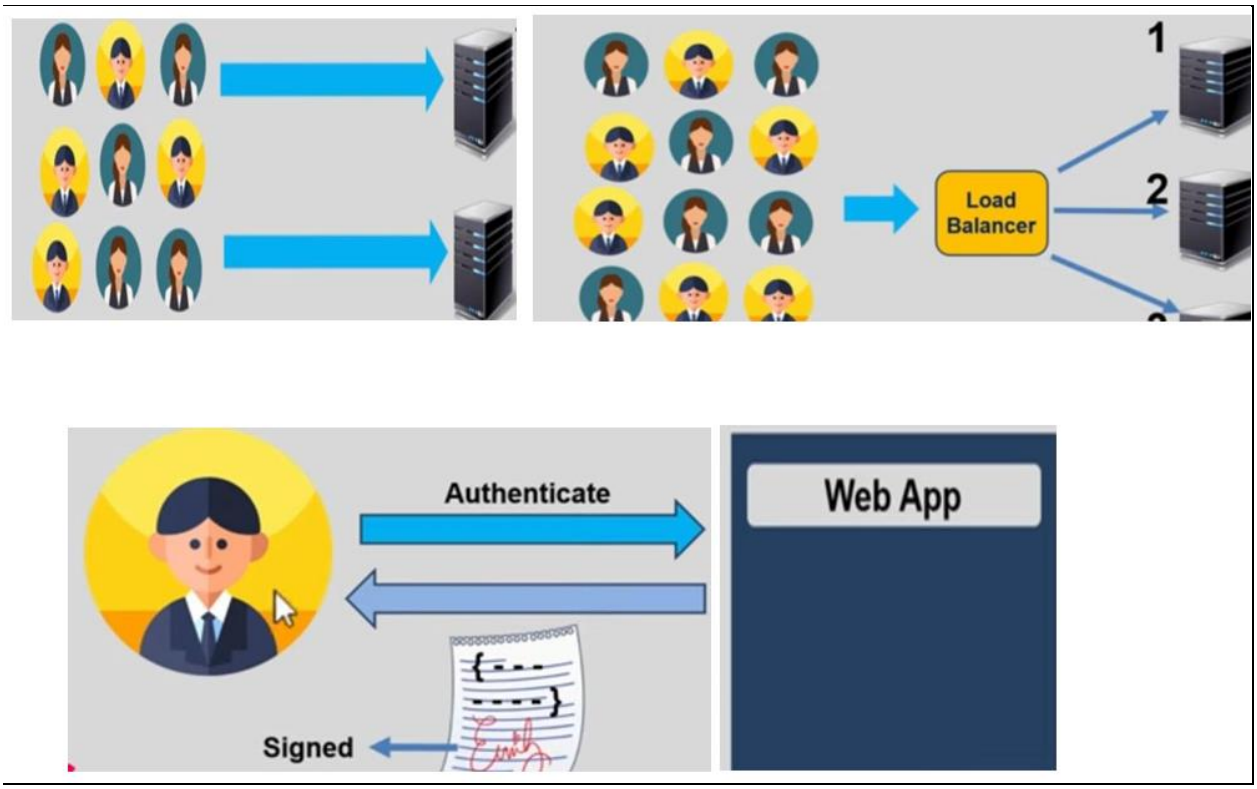
Advantages of JWT: -

- Unlike sessions, JWTs are stateless and do not require server-side storage.
- They can be used across multiple servers and domains.
- They provide a secure way of sharing authentication data.

15.7. Conclusion

Understanding HTTP's stateless nature, session-based authentication, and JWT

based authentication helps in designing efficient and scalable web applications. While sessions provide a traditional way to maintain user state, JWTs offer a more flexible, scalable, and secure alternative.



15.8 Practical implementation:-

1. Create a folder **& run Npm init command** in the terminal of created folder

2. **Install all** require packages such as **express,jsonwebtokens**

3. Create a file named as **index.js**

```
const express = require("express");
const jwt = require("jsonwebtoken");
const app = express();
const secretkey = "My@31";
app.get("/", (req, res) => {
   res.json({
      message: "authentication testing"
```

```
    });
});
app.post("/login", (req, res) => {
    const user = {
        id: 1,
        name: "backend",
        email: "backend@gmail.com"
    };
    jwt.sign({ user }, secretkey, { expiresIn: '300s' },
(err, token) => {
        res.json({
            token
        });
    });
});
app.post("/profile", verifyT, (req, res) => {
    jwt.verify(req.token, secretkey, (err, authData) =>
{
        if (err) {
            res.send({ result: "invalid token" });
        } else {
            res.json({
                message: "profile sent successfully",
                authData
            });
        }
    });
```

```
});

function verifyT(req, res, next) {
    const getHeader = req.headers['authorization'];
    if (typeof getHeader !== 'undefined') {
        const bearer = getHeader.split(" ");
        const token = bearer[1];
        req.token = token;
        next();
    } else {
        res.send({ result: "token not valid" });
    }
}

app.listen(3000, () => console.log("app running on
port 3000"));
```

16)Using Passport.js for Authentication

Passport is a popular, modular authentication middleware for Node.js applications. With it, authentication can be easily integrated into any Node- and Express-based app. The Passport library provides more than 500 authentication mechanisms, including OAuth, JWT, and simple username and password-based authentication.

Using Passport makes it easy to integrate more than one type of authentication into the application too. It's flexible, modular, and supports multiple authentication strategies like:

- Local username/password
- OAuth (Google, GitHub, Facebook, etc.)
- JWT-based auth
- Session-based login

Passport simplifies the process of managing login and protected routes across APIs and full-stack apps.

16.1 Installing Passport and Local Strategy

```
npm install passport passport-local express-session bcryptjs
```

16.2 project using passport JS

Folder Structure (Basic)

```
passport-local-app/
├── models/
│   └── User.js
├── config/
│   └── passport.js
├── app.js
├── .env
├── package.json
```

Step 1:- models/User.js – Mongoose User Schema

```
const mongoose = require('mongoose');

const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({

  username: { type: String, unique: true },

  password: { type: String }

});

// Hash password before saving

userSchema.pre('save', async function (next) {

  if (!this.isModified('password')) return next();
```

```
  this.password = await bcrypt.hash(this.password,
10);

  next();

});

// Compare password method

userSchema.methods.matchPassword = function
(enteredPassword) {

  return bcrypt.compare(enteredPassword,
this.password);

};

module.exports = mongoose.model('User',
userSchema);
```

Step 2:- config/passport.js – Configure Passport Local Strategy

```
const LocalStrategy = require('passport-local').Strategy;

const User = require('../models/User');

module.exports = function (passport) {

  passport.use(

    new LocalStrategy(async (username, password, done)
=> {

      try {

        const user = await User.findOne({ username });

        if (!user) return done(null, false, { message: 'User not
found' });

        const isMatch = await user.matchPassword(password);

        if (!isMatch) return done(null, false, { message: 'Wrong
password' });

        return done(null, user);
```

```
    } catch (err) {
      return done(err);
    }
  })
);

// Serialize user to store in session
passport.serializeUser((user, done) => {
  done(null, user.id);
});

// Deserialize user from session
passport.deserializeUser(async (id, done) => {
  try {
    const user = await User.findById(id);
    done(null, user);
  } catch (err) {
    done(err, null);
  }
});
};
```

Step 3:- app.js – Express Setup with Passport

```
const express = require('express');
const mongoose = require('mongoose');
const passport = require('passport');
```

```
const session = require('express-session');
const User = require('./models/User');
require('dotenv').config();
require('./config/passport')(passport);
const app = express();
// Middleware
app.use(express.json());
app.use(express.urlencoded({ extended: false }));

// Session setup
app.use(
  session({
    secret: 'secret-key',
    resave: false,
    saveUninitialized: false
  })
);

// Passport middleware
app.use(passport.initialize());
app.use(passport.session());

// MongoDB connection
mongoose.connect('mongodb://localhost:27017/passport-demo');
```

```
// Routes
app.post('/register', async (req, res) => {
  const { username, password } = req.body;
  try {
    const user = new User({ username, password });
    await user.save();
    res.json({ message: 'User registered' });
  } catch (err) {
    res.status(400).json({ error: err.message });
  }
});


app.post('/login',
  passport.authenticate('local', {
    failureRedirect: '/login-failed',
    successRedirect: '/dashboard'
  })
);
app.get('/dashboard', isAuthenticated, (req, res) => {
  res.send(`Welcome, ${req.user.username}`);
});
app.get('/logout', (req, res) => {
  req.logout(() => res.send('Logged out successfully'));
});
function isAuthenticated(req, res, next) {
  if (req.isAuthenticated()) return next();
```

```
  res.status(401).send('Not authorized');
}
app.get('/login-failed', (req, res) => {
  res.send('Login failed. Invalid credentials.');
});
app.listen(3000, () => {
  console.log('Server running on http://localhost:3000');
});
```

Session-Based Login Testing:- Use **Postman** or browser forms to:

Endpoint	Method	Body
/register	POST	{ "username": "admin", "password": "1234" }
/login	POST	Form data with username & password
/dashboard	GET	Requires active session
/logout	GET	Logs user out and clears session

17)API Development

API (Application Programming Interface) allows communication between two software components. In web development, APIs are typically built using HTTP methods to enable client applications (like frontend apps or mobile apps) to interact with server data. Whether you're building a web app, or mobile service, or managing complex data, learning how to build an API is essential for creating scalable, efficient systems. APIs can be categorized into several types based on their architecture, such as REST, Graph-QL, and SOAP, each with specific use cases.

17.1 Creating Secure and Scalable APIs

1. **Planning Your API**
    - **Define the Purpose:-** Determine what your API will do. Examples:
        - Shopping cart API
        - Weather information provider
        - Social media interaction handler
    - **Identify Resources:- APIs operate around resources like**
        - Users
        - Products
        - Orders
    - **Define Endpoints and Methods:- Use REST principles to assign actions:**
        - GET /api/v1/products
        - POST /api/v1/orders
        - PUT /api/v1/users/:id
        - DELETE /api/v1/orders/:id

2. **API Design**
    - **Choose Your Architectural Style**
        - **REST**: HTTP-based, stateless, resource-focused.
        - **SOAP**: XML-based, security heavy.
        - **GraphQL**: Flexible querying, avoids over-fetching.
    - **API Requirements**
        - Authentication **(e.g. JWT)**

        - Rate Limiting
        - Data Format**: Usually JSON**
        - Versioning**: /api/v1/...**
3. **Set Up Your Development Environment**
    - Node.js with Express
4. **Building API Endpoints**
    - Node.js with Express
5. **Securing Your API**
    - Node.js with JWT
6. **Testing Your API**
    - **Manual Testing with Postman:-**Manually test endpoints.
        - Include headers, body, and auth tokens.
    - **Automated Testing (Node.js (Mocha + Chai)**
        - npm install mocha chai chai-http --save-dev
        - chai.request(app).get('/api/v1/products')...
7. **Monitoring and Optimizing Your API**
    - **Monitoring API Performance**
        - **Heroku Metrics**: Built-in CPU/memory stats.
        - **AWS CloudWatch**: Real-time logs and alerts.
    - **Optimizing API Performance**
        - Enable Gzip Compression
        - Apply Caching (e.g., Redis)
        - Add Pagination for large datasets
        - Implement Rate Limiting

18. RESTful API

A RESTful API (Representational State Transfer) is an architectural style that uses standard HTTP methods to operate on resources (data entities) exposed via URL endpoints. RESTFUL APIs:

- Are stateless (each request is independent)
- Work over HTTP/HTTPS
- Use standard methods: GET, POST, PUT, DELETE
- Operate on resources (nouns) identified via URIs
- Return data in commonly used formats (e.g., JSON)

18.1 Core REST Principles

Principle	Description
Client-Server	Separation of UI (frontend) and data logic (backend)
Stateless	No session state is stored server-side between requests
Cacheable	Responses can be cached to improve performance
Uniform Interface	Use consistent URL patterns and HTTP methods

18.2 HTTP Status Codes in REST APIs

Code	Meaning	When to Use
200	OK	Successful GET, PUT, PATCH requests
201	Created	POST request success
204	No Content	Successful DELETE
400	Bad Request	Invalid data
401	Unauthorized	Missing or invalid auth token

403	Forbidden	Authenticated but lacks permissions
404	Not Found	Resource doesn't exist
500	Internal Server Error	Unexpected server issue

18.3 practical setup

Step 1:- Open your terminal & type the command

- **mkdir restful-api-example && cd restful-api-example**
- **npm init -y**
- **npm install express**

Step 2:- create a file named as **app.js**

```
const express = require('express');

const app = express();

app.use(express.json());

// Sample in-memory data

let products = [

  { id: 1, name: 'Laptop', price: 1000 },

  { id: 2, name: 'Phone', price: 500 }

];

// GET all products

app.get('/api/v1/products', (req, res) => {

  res.status(200).json(products);

});

// GET product by ID

app.get('/api/v1/products/:id', (req, res) => {
```

```
  const product = products.find(p => p.id === parseInt(req.params.id));
  if (!product) return res.status(404).json({ message: 'Product not found' });
  res.json(product);
});
// POST new product
app.post('/api/v1/products', (req, res) => {
  const newProduct = {
    id: products.length + 1,
    name: req.body.name,
    price: req.body.price
  };
  products.push(newProduct);
  res.status(201).json(newProduct);
});
// PATCH update product
app.patch('/api/v1/products/:id', (req, res) => {
  const product = products.find(p => p.id === parseInt(req.params.id));
  if (!product) return res.status(404).json({ message: 'Product not found' });
  if (req.body.name) product.name = req.body.name;
  if (req.body.price) product.price = req.body.price;
  res.json(product);
});
// DELETE product
```

```
app.delete('/api/v1/products/:id', (req, res) => {
  products = products.filter(p => p.id !== parseInt(req.params.id));
  res.status(204).send();
});

app.listen(3000, () => console.log('API running on http://localhost:3000'));
```

19. API Versioning and Documentation

19.1. API Versioning

- Prevents breaking changes for existing users
- Allows smooth transition from old to new features
- Enables iterative improvements without disrupting clients

19.2 Versioning Approaches

Method	Example	Notes
URI Versioning	/api/v1/products	Most common and clear
Header Versioning	Accept: application/vnd.api.v2+json	Clean URLs, but harder to debug
Query Params	/api/products?version=2	Easy to implement, not RESTful

19.3 Folder Structure (URI Versioning)

routes/

├— v1/

│ └── users.js

├── v2/

│ └── users.js

19.4 Express.js Routing Example

```
app.use('/api/v1/users', require('./routes/v1/users'));

app.use('/api/v2/users', require('./routes/v2/users'));
```

Best Practice: Always include versioning in URI unless you have a good reason not to.

19.5 API Documentation

- Helps frontend, mobile, and 3rd-party developers understand how to use your API
- Makes onboarding faster and easier
- Acts as a contract between client and server
- Essential for public APIs

19.6 Common Tools for API Documentation

Tool	Use Case
Swagger / OpenAPI	Auto-generate docs with testing UI
Postman	Manual collections and sharing
Redoc	Converts OpenAPI specs into clean UI
apidoc.js	Generates docs from inline comments

19.7 Swagger + Express.js Setup Example

Step 1. Install dependencies

```
npm install swagger-ui-express swagger-jsdoc
```

Step 2. Configure Swagger in swagger.js

```
const swaggerJsdoc = require('swagger-jsdoc');

const swaggerUi = require('swagger-ui-express');

const options = {

 definition: {

   openapi: '3.0.0',

   info: {

     title: 'Product API',

     version: '1.0.0',

   },

 },

 apis: ['./routes/v1/*.js'],

};

const specs = swaggerJsdoc(options);

module.exports = { swaggerUi, specs };
```

Step 3. Use in app.js

```
const express = require('express');

const app = express();

const { swaggerUi, specs } = require('./swagger');

app.use('/api-docs', swaggerUi.serve,
swaggerUi.setup(specs));
```

Step 4. Add Comments in Routes

```
/**
 * @swagger
 * /api/v1/products:
 *   get:
 *     summary: Retrieve all products
 *     responses:
 *       200:
 *         description: A list of products
 */
router.get('/products', (req, res) => { ... });
```

20. Real-time communication

Real-time communication refers to the immediate or near-immediate exchange of data between clients and servers without needing the client to continuously poll the server for updates. common real-time use cases:

- Chat applications
- Live sports updates
- Online gaming
- Live stock prices
- Collaborative tools (e.g., Google Docs)

20.1 How Real-time Communication Works

Instead of a request-response model (as in traditional APIs), real-time apps establish a **persistent connection** using:

- **WebSockets**: Full-duplex communication between client and server
- **Socket.IO**: A wrapper over WebSockets with fallback support and more features
- **Server-Sent Events (SSE)**: Server pushes updates to clients (one-way)
- **Long Polling**: Emulates real-time using repeated HTTP requests

21. Introduction to WebSockets

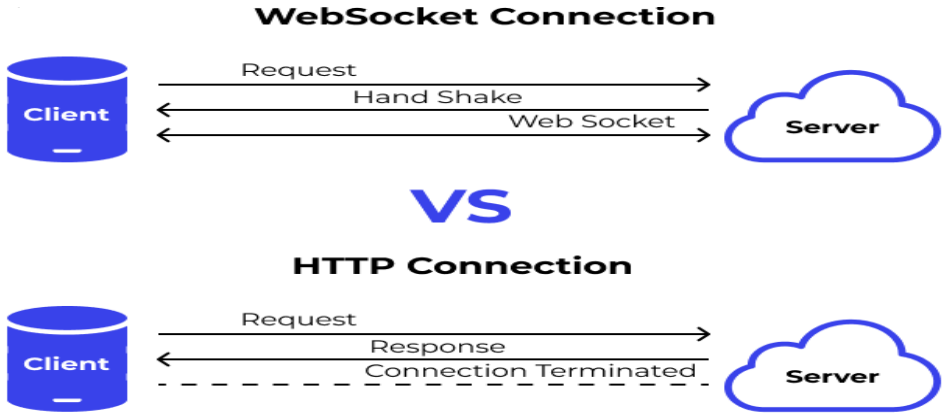
Web-Sockets are a protocol that enables **full-duplex communication** between a client and server over a single, long-lived TCP connection. That means that the client no longer needs to be an initiator of a transaction while requesting data from both the server and database.

Unlike HTTP, which follows a request-response model, Web-Sockets allow real-time, bidirectional communication without the overhead of multiple HTTP requests.

> **With WebSockets, during the first request to the backend server, except receiving incoming data, the socket.io client also establishes an initial connection. This process is known as the *WebSocket handshake*.** 🤝

21.1 How Web-Sockets Work

- A client sends an HTTP handshake to upgrade the connection.
- The server accepts the upgrade and switches the protocol.
- A **persistent TCP connection** is established.
- Both client and server can now send messages to each other at any time.



21.2 WebSocket Lifecycle

Phase	Description
Handshake	Client sends a GET request to upgrade
Open	Persistent connection is established
Message	Data is exchanged in both directions
Close	Either party can close the connection
Error	Connection error or unexpected closure

21.3 Does it mean it's impossible to create a real-time chat application without WebSockets?

Ans:-Well, there is a technique **called HTTP Long Polling**. Using this, the client sends a request and the server holds that opened until some new data is available. As soon as data appears and the client receives it, a new request is immediately sent and operation is repeated over and over again. However, a pretty big disadvantage of using HTTP Long Polling is that it consumes a lot of server resources**. To put it in other words:**

> WebSockets are Full-Duplex, while HTTP Long Polling is Half-Duplex. In the case of the former both the client and the server send and receive messages. As for HTTP Long Polling, a new request-response cycle is necessary each time the client attempts to communicate with the server.

The Socket io library is a JavaScript library that enables real-time, bidirectional, event-based communication between the connected clients (browser) and the server side. It consists of a JavaScript client library and another one dedicated to servers. Both components share most of their API.

22. using Socket.io Building a Real-time Chat Application

Step 1: Make a folder named as **project** & open terminal from root directory then type the command. This will create a package.json file.

```
npm init
```

Step 2: Now run the below command to install express Js

```
npm install express
```

Step 3: Check in package.json if all the dependences has been installed or not.

Step 4: Now make an **index.js** file in the **project** folder.

index.js

```
const http=require("http");
const express=require("express");
const path=require("path");
const app=express();
const server=http.createServer(app);
app.use(express.static(path.resolve("./public")));
app.get("/",(req,res)=>{
```

```
    return res.sendFile("./public/index.html");
});
server.listen(9000,()=>console.log(`server started at port :9000`));
```

Step 5: create a **public folder** inside project folder & make **index.html** file inside it.

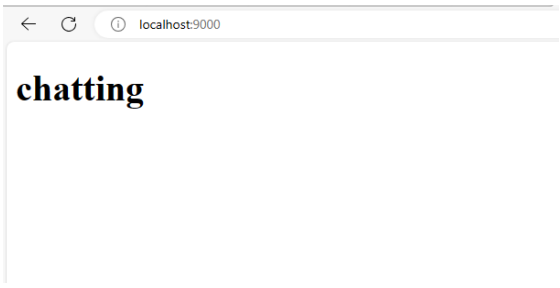
index.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>my chat app</title>
    </head>
<body>
    <h1>chatting</h1>
</body>
```

Step 6: Now run the files

Run: node index.js

Output:



Step 7: Now install the **socket.io** package so make socket connections by using the command:

```
npm install socket.io
```

Now updated version of index.js is

```
const http=require("http");
const express=require("express");
const path=require("path");
const { Server } = require("socket.io");
const app=express();
const server=http.createServer(app);
const io= new Server(server);
//socket.io
io.on("connection", (socket) =>{
   console.log(" a new user has connected",socket.id);
});
app.use(express.static(path.resolve("./public")));
app.get("/",(req,res)=>{
   return res.sendFile("./public/index.html");
});
server.listen(9000,()=> console.log(`server started at port :9000`));
```

updated index.html

```
<!DOCTYPE html><html>
   <head>
      <title>my chat app</title>
   </head>
<body>
```

```
    <h1>chatting</h1>
    <script src="/socket.io/socket.io.js"></script>
</body>
```

Now run the file again
Run: node index.js

Output: Now inspect the output page & go to network tab & check for the url, you will be able to see the long script.

Now update index.js

Now update index.html

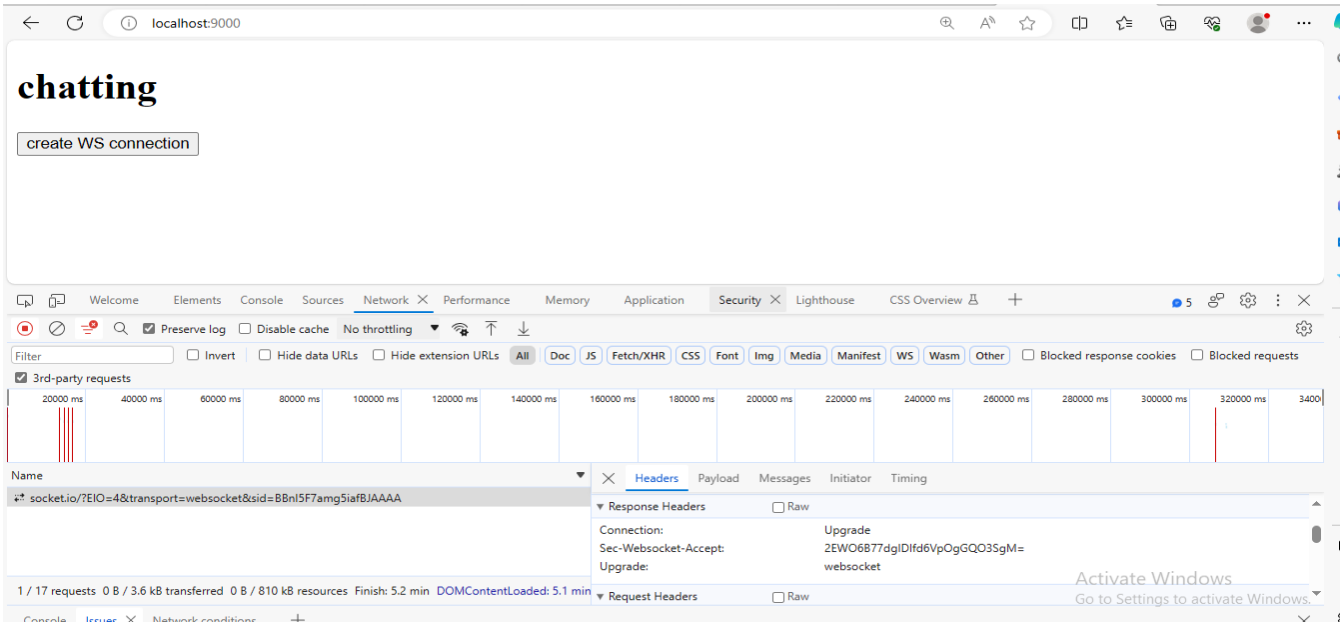
Let's create a button so that after clicking the button a **ws** must be created which will make http request & will have an **upgrade header.**

Index.html

```
<!DOCTYPE html>
<html>
    <head>
        <title>my chat app</title>
    </head>
<body>
    <h1>chatting</h1>
    <button onclick="createConnection()">create WS
connection</button>
    <script src="/socket.io/socket.io.js"></script>
    <script>
        function createConnection(){
            const socket=io();
```

```
    }

  </script></body>
```

Run the file again: node index.js



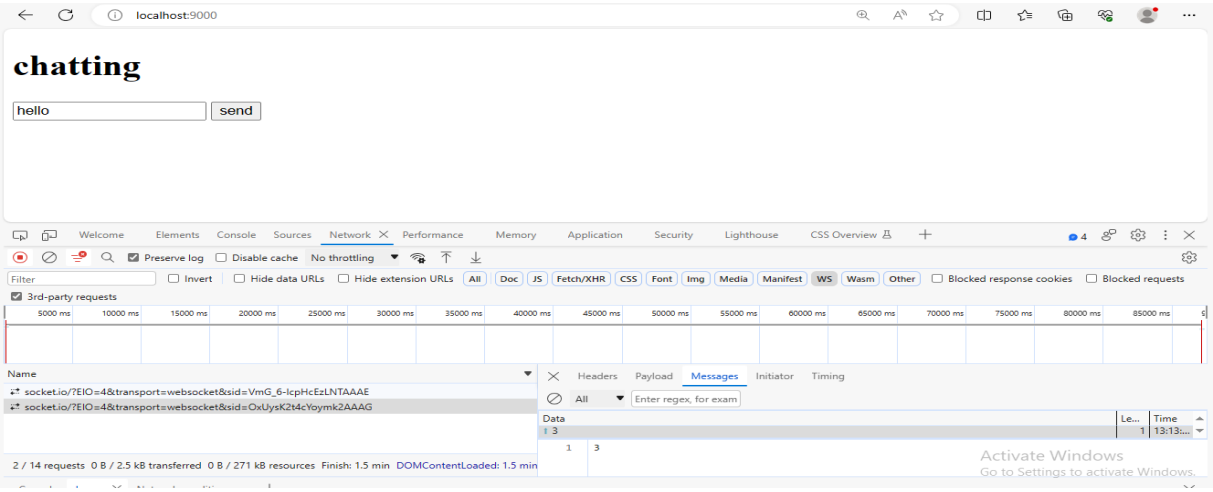
Updated index.html

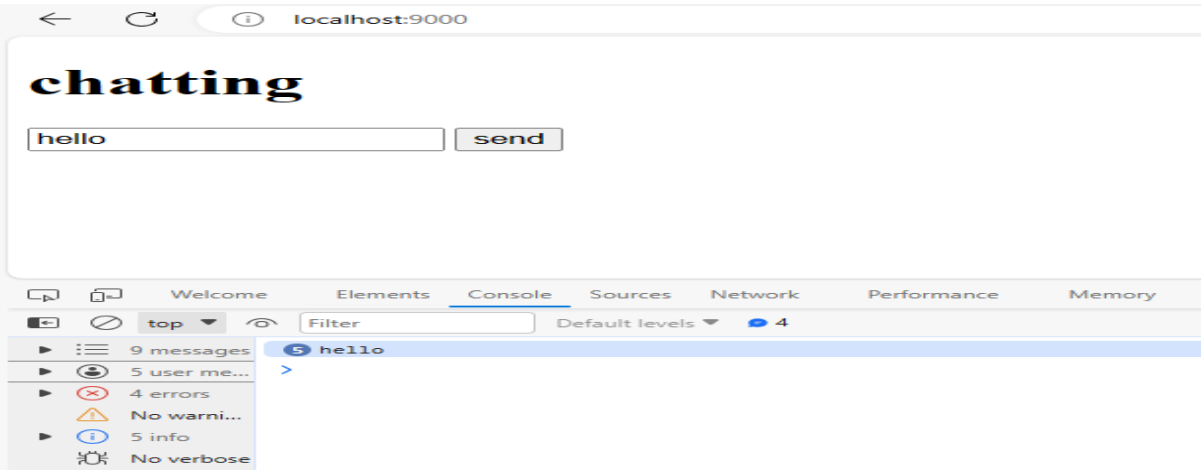
```
<!DOCTYPE html>

<html>

  <head>

    <title>my chat app</title>

  </head>

<body>

  <h1>chatting</h1>

  <input type="text" id="message" placeholder="enter message"/>

  <button id="sendBtn">send</button>

  <script src="/socket.io/socket.io.js"></script>

  <script>
```

```
    const socket=io();
    const sendBtn=document.getElementById("sendBtn");
    const messageInput=document.getElementById("message");
    sendBtn.addEventListener('click' ,e => {
        const message = messageInput.value;
        console.log(message);
    })
  </script>
</body>
```

Run again

```
PS C:\Users\DELL\Desktop\BackEnd\sockettest> node index.js
server started at port :9000
 a new user has connected JlBDRaOowOPN5LUmAAAB
 a new user has connected ZGdhvL8V-moAiqQOAAAD
 a new user has connected _xtKP43sHOuSSSd6AAAF
```



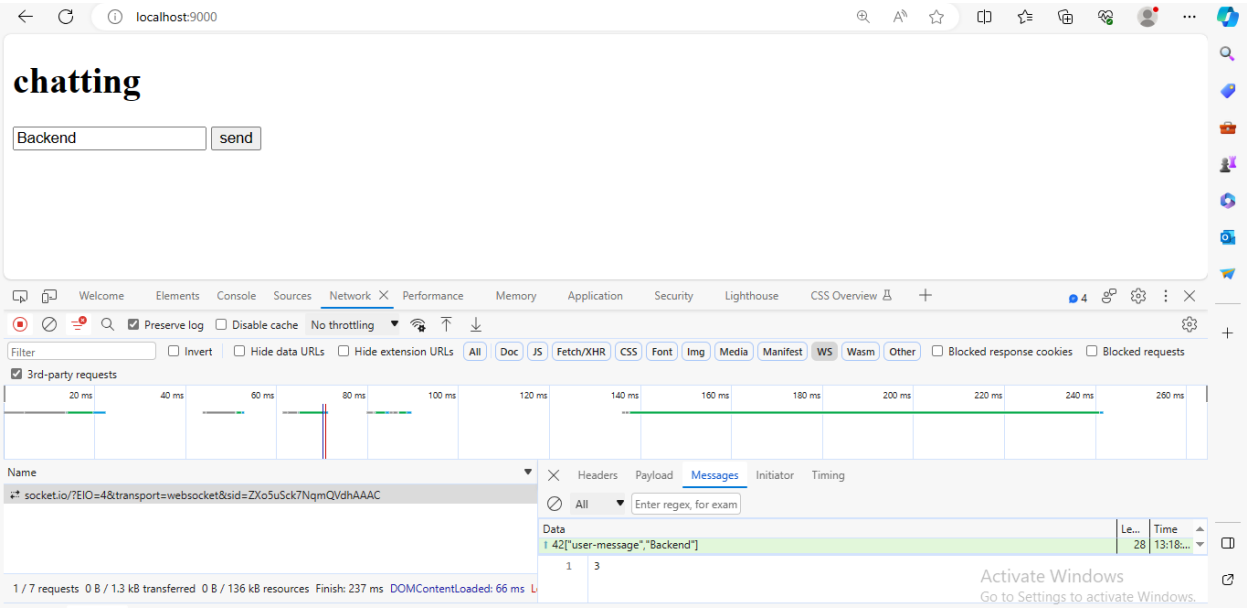


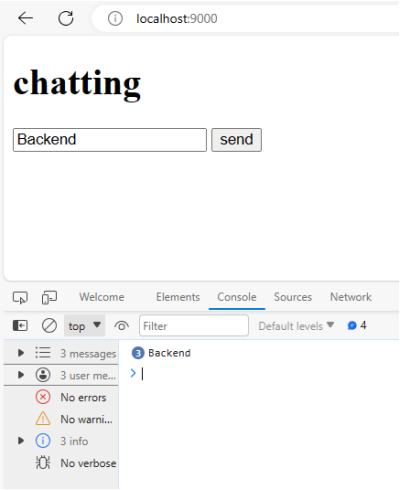
Updated index.html

```
<!DOCTYPE html>

<html>

  <head>

    <title>my chat app</title>

  </head>

<body>

  <h1>chatting</h1>

  <input type="text" id="message" placeholder="enter message"/>

  <button id="sendBtn">send</button>

  <script src="/socket.io/socket.io.js"></script>

  <script>

    const socket=io();

    const sendBtn=document.getElementById("sendBtn");

    const messageInput=document.getElementById("message");

    sendBtn.addEventListener('click' ,e => {

      const message = messageInput.value;

      console.log(message);
```

```
        socket.emit('user-message',message);

    })

  </script>

</body>
```

Run again: Node index.js





Index.js

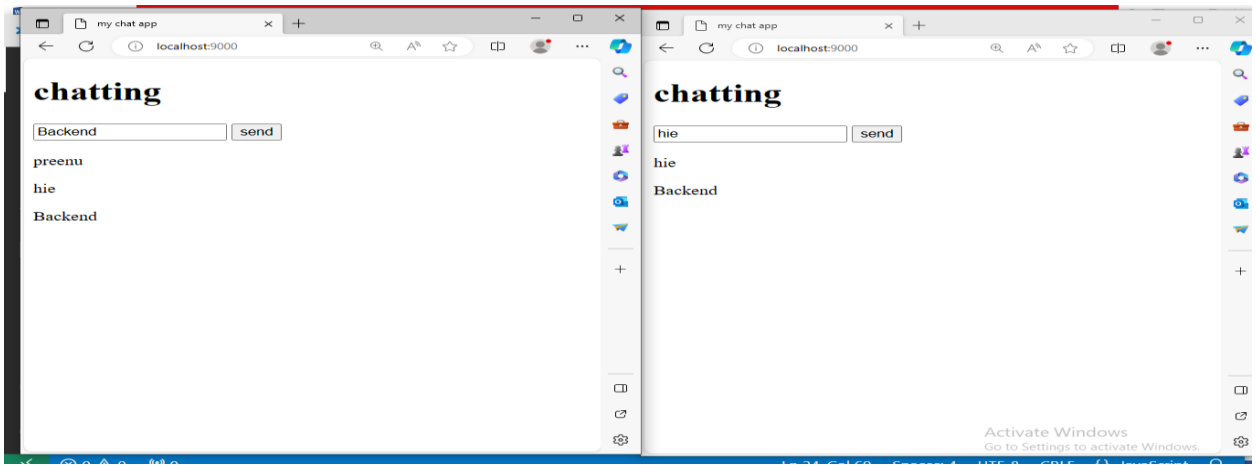
```
const http=require("http");
const express=require("express");
const path=require("path");
const { Server }=require("socket.io");
const app=express();
const server=http.createServer(app);
const io=new Server(server);


//socket.io
io.on("connection", (socket) => {
   socket.on("user-message", (message) =>{
      //console.log("new user message",message);
      io.emit("message",message);
   });
});
app.use(express.static(path.resolve("./public")));
app.get("/",(req,res)=>{
   return res.sendFile("./public/index.html");
});
server.listen(9000,()=>console.log(`server started at port :9000`));
```

index.html

```
<!DOCTYPE html>
<html>
   <head>
```

```
    <title>my chat app</title>
  </head>
<body>
  <h1>chatting</h1>
  <input type="text" id="message" placeholder="enter message"/>
  <button id="sendBtn">send</button>
  <div id="messages"></div>
 <script src="/socket.io/socket.io.js"></script>
  <script>
    const socket=io();
    const sendBtn=document.getElementById("sendBtn");
    const messageInput=document.getElementById("message");
    const allMessages=document.getElementById("messages");
    socket.on('message', (message) => {
      const p=document.createElement("p");
      p.innerText=message;
      allMessages.appendChild(p);
    });
    sendBtn.addEventListener("click",(e) => {
      const message=messageInput.value;
      console.log(message);
      socket.emit("user-message",message);
    });
  </script>
</body>
```

RUN: node index.js



23. Deploying Backend APIs Using Render.com

Render is a modern cloud platform that allows developers to deploy full-stack applications with minimal configuration. It supports static sites, backend services, cron-jobs, and databases — and is an excellent option for deploying Node.js APIs.

Feature	Benefit
Free Tier Available	Great for small projects & learning
Easy Git Integration	Auto-deploy from GitHub/GitLab
HTTPS by Default	Secure connection without setup
Environment Variables	Manage secrets securely
Built-in Monitoring	See logs, performance, and errors

23.1 Prerequisites

- GitHub account with your backend API pushed to a public or private repo
- Node.js project with a package.json and entry file (e.g., app.js or server.js)

23.2 Step-by-Step Deployment Guide

Step 1. Prepare Your Node.js App:- Ensure you

- Have an app.listen() on a port from process.env.PORT

```
const PORT = process.env.PORT || 3000;

app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

- Push your project to GitHub

Step 2. Create a New Web Service on Render

1. Go to https://dashboard.render.com
2. Click New → Web Service
3. Connect your GitHub repo
4. Configure the service:
    - Name: my-api
    - Environment: Node
    - Build Command: npm install
    - Start Command: node app.js (or your entry file)
    - Branch: main or master

Step 3. Add Environment Variables (Optional):- If using .env variables:

1. Go to Environment > Add Environment Variable
2. Add entries like:
    - MONGO_URI
    - JWT_SECRET

Step 4. Trigger Deployment:- Once set up, Render will:

- Install dependencies
- Run your build command

- Start the app on a public URL (e.g., https://my-api.onrender.com)

Each push to your GitHub repo will auto-deploy.

23.3 Updating Your API

Every commit to your connected GitHub branch will trigger an automatic deployment. You can also manually trigger a deploy from the Render dashboard.

23.4 Useful Tips

- Render supports custom domains, automatic SSL, and PR previews.
- Use **render.yaml** for Infrastructure as Code **(IaC).**
- Use logs tab in dashboard for debugging errors.

Example package.json

```
{
  "name": "my-api",
  "version": "1.0.0",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```